KU LEUVEN

Chapter 3. Predictive Analytics: Decision Trees

Data Science for Business

D0I74A

Prof. dr. Wouter Verbeke



Course schedule

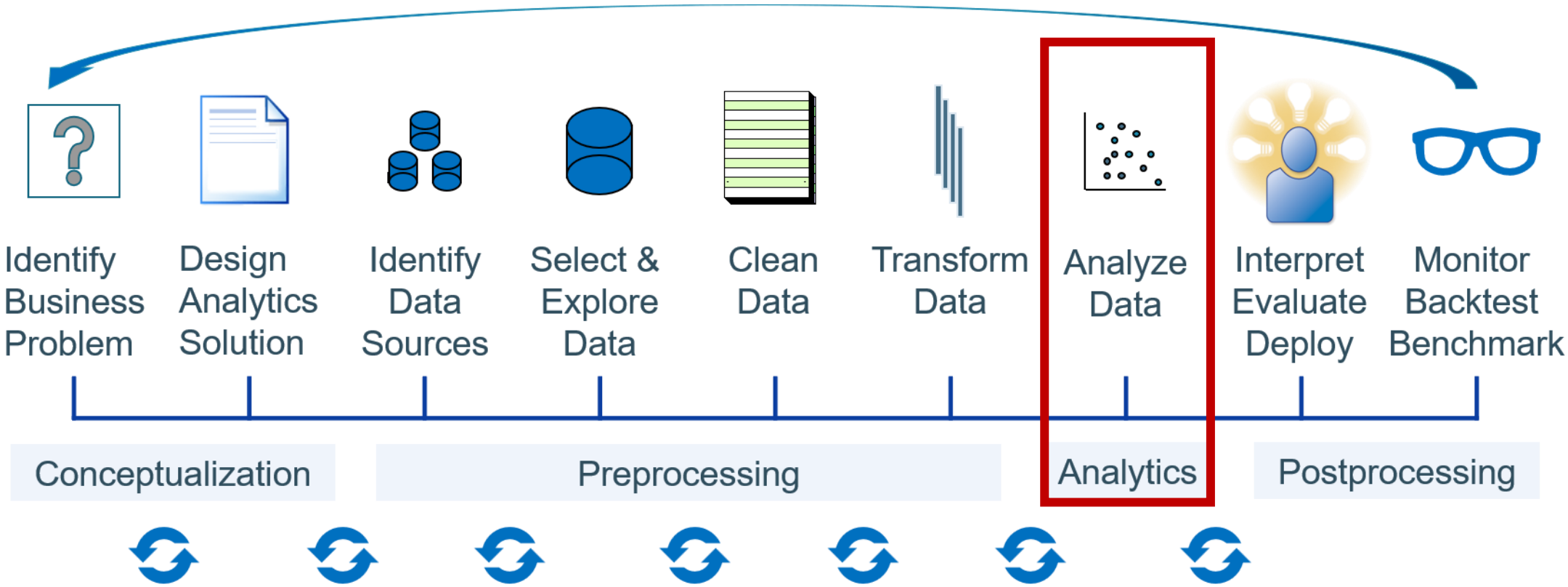
BLOCK1

1	21/09/2026	Chapter 0. Course Outline Chapter 1. Data Science for Business	
1	22/09/2026	Chapter 1. Data Science for Business (ctd.)	
		Hands-on #1: Building a first pipeline	**Homework 1**
2	28/09/2026	Chapter 1. Data Science for Business (ctd.) Chapter 2. Exploratory Data Analysis & Preprocessing	
2	29/09/2026	Chapter 2. Exploratory Data Analysis & Preprocessing (ctd.)	
		Hands-on #2: EDA & Preprocessing	**Homework 2**
3	05/10/2026	Chapter 2. Exploratory Data Analysis & Preprocessing (ctd.) Chapter 3. Predictive Analytics - Decision Trees	
3	06/10/2026	Chapter 4. Predictive Analytics - Nearest Neighbors & Ensembles	
		Hands-on #3: Predictive models	**Homework 3**
4	12/10/2026	Chapter 5. Postprocessing	
4	13/10/2026	Chapter 5. Postprocessing	
		Hands-on #4: Postprocessing	**Homework 4**

Data science process model

Chapter 3

- Practical steps in elaborating a customer churn prediction model:



Outline

1. Classification trees
2. Re-sampling
3. Regression trees
   - Self-study: ID3 with Standard Deviation Reduction

Learning objectives

At the end of this chapter, students should be able to:

- Understand and explain the **workings of decision tree algorithms** (classification and regression) for learning trees from data (e.g., information gain, entropy, pruning, SDR, etc.)
- **Interpret and explain a decision tree model** as obtained when applying a decision tree method on data
- Discuss the **uses**, **advantages** and **disadvantages** of decision trees, in comparison with alternative predictive methods (cfr. subsequent chapers)

1. Classification trees

Introductory example

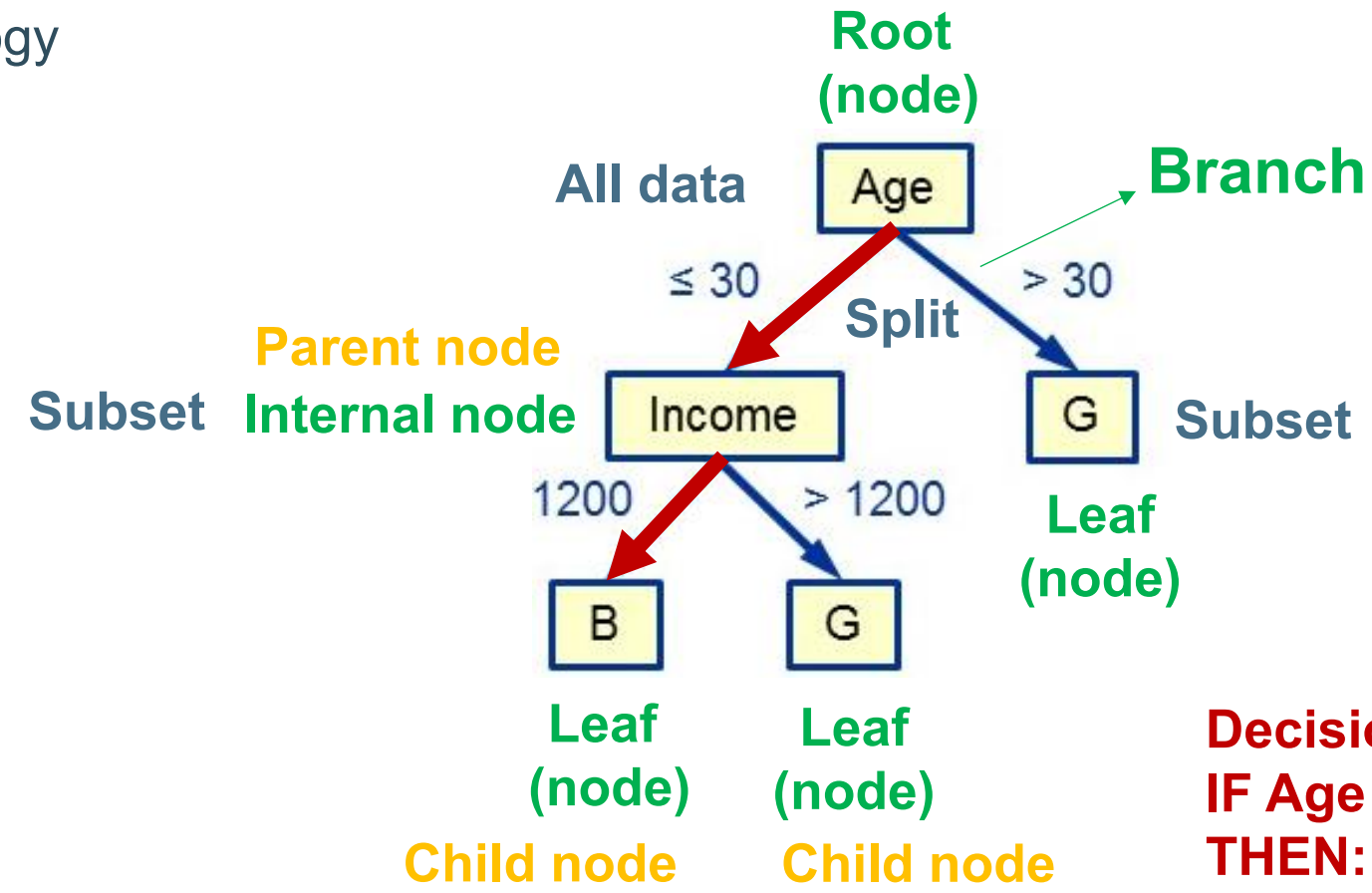
Target variable: Default

ID	Age	Income	Duration	…	Default
12580	45	2600	22	…	Good
42882	25	1100	1	…	Bad
42440	27	1400	15	…	Good
24580	57	3200	37	…	Good
31008	32	1800	6	…	Good
21007	42	1600	0	…	Good
67088	21	1900	0	…	Bad
45880	24	1500	12	…	Good
…	…	…	…	…	…

FYI: **Duration** or service duration: how many years a customer has been using services (e.g., by a financial institution), across products (e.g., accounts, cards, loans, insurance, …)

Introductory example

- Terminology

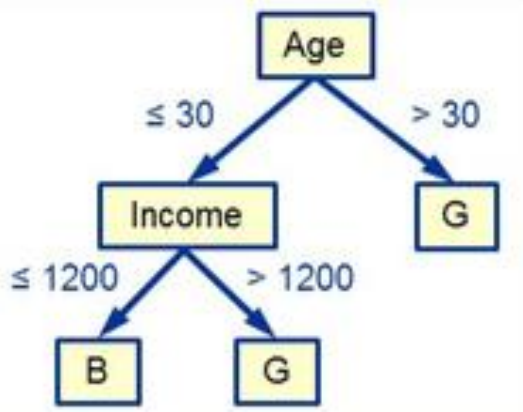


Decision rule:
IF Age =< 30 AND Income =< 1200
THEN: Default = Bad

Introductory example

- Various **decision tree learners** exist *to induce* a tree from data:
  - ID3, C4.5, See5 (R. Quinlan 1986, 1993)
    - https://cis.temple.edu/~ingargio/cis587/readings/id3-c45.html
  - CART: classification and regression trees (Breiman et al., 1984)
  - CHAID: chi-squared automatic interaction detection (Hartigan, 1975)

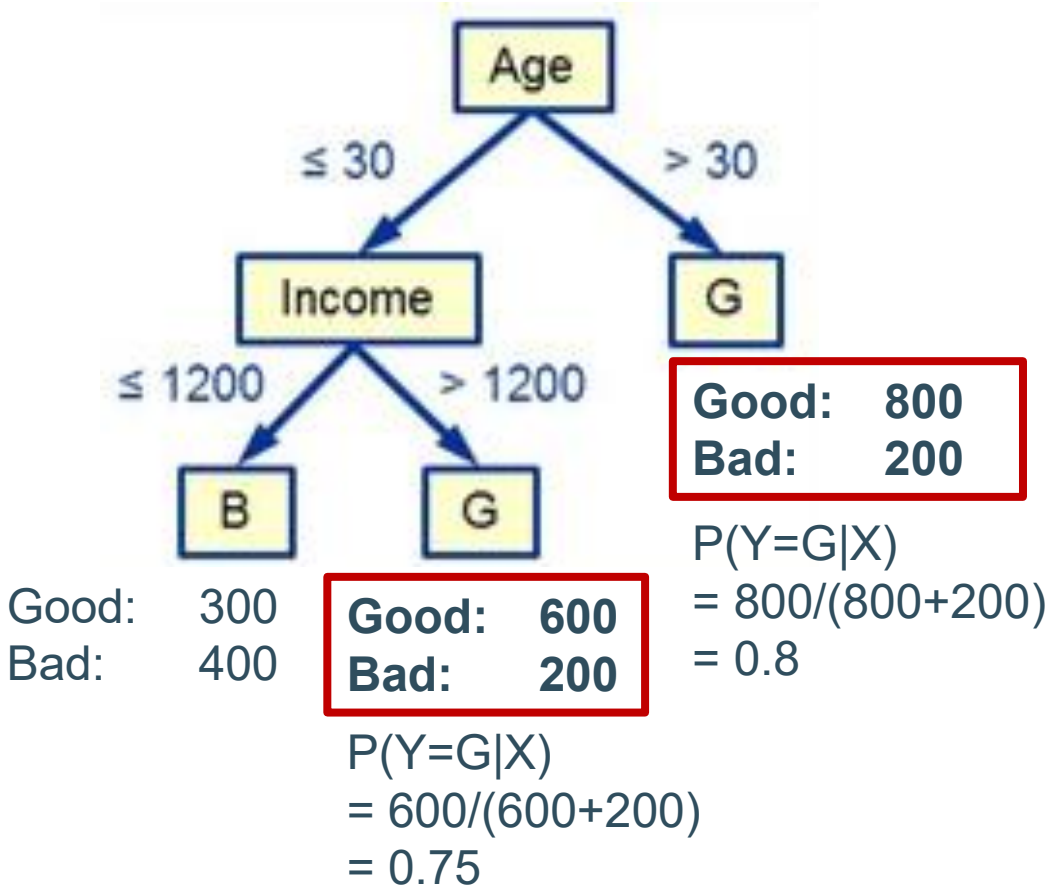
- What kinds of decisions are to be made in *learning* a tree?
  1. **Splitting decision**
  2. **Stopping decision**
  3. **Assignment decision**



Assignment decision

- Simple: majority voting
  - Class of most of the observations in a leaf

- Better: class probability
  - **Why better?**
  - Differentiation!
  - Flexibility towards decision making!



Splitting decision

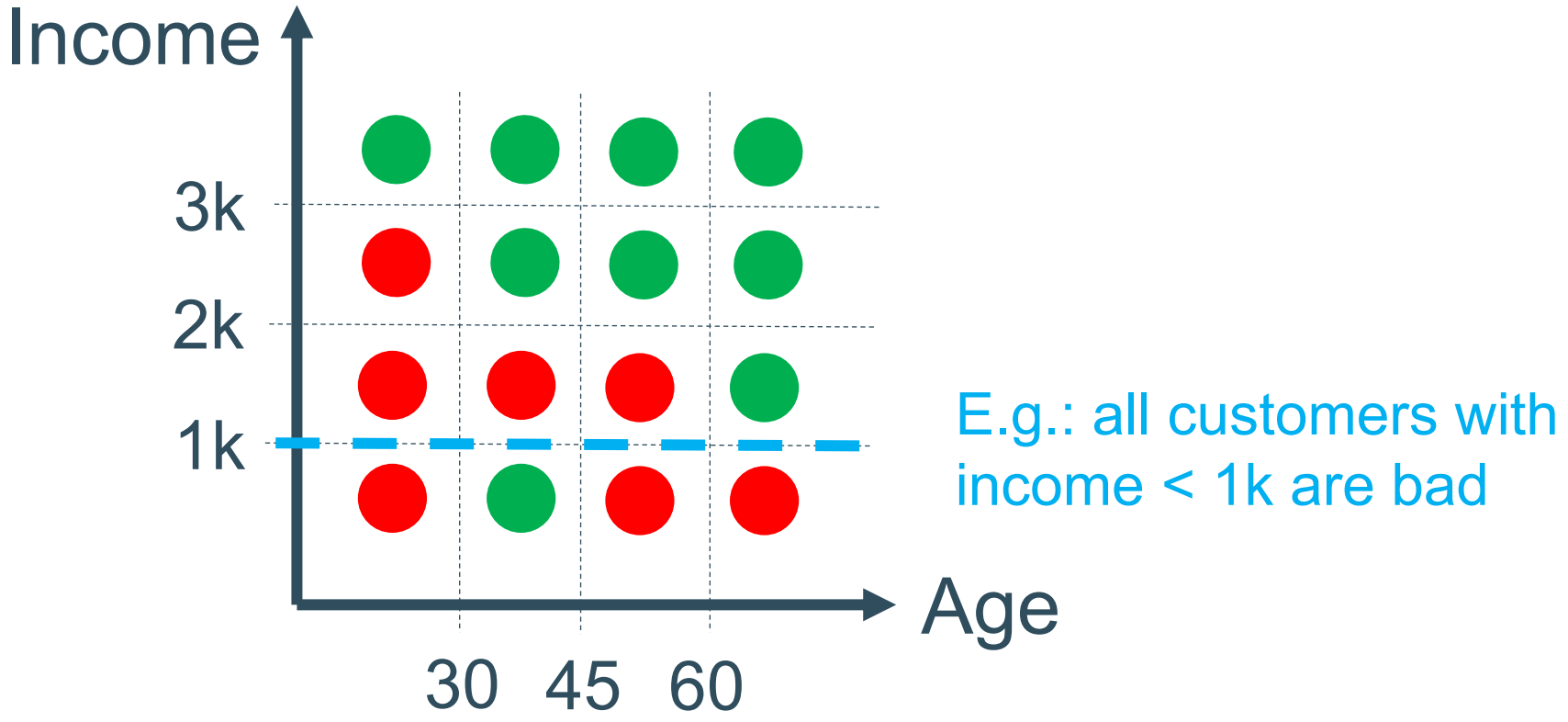
- Decision tree learning strategy:
  - The sample is iteratively split in smaller samples
  - Each time all possible splits are evaluated
  - The best possible split is selected

Why iteratively?

How do we define ‘best’?

Splitting decision

- How would you split the following set of examples to separate goods (green) from bads (red) making **a single split?**



E.g.: all customers with income < 1k are bad

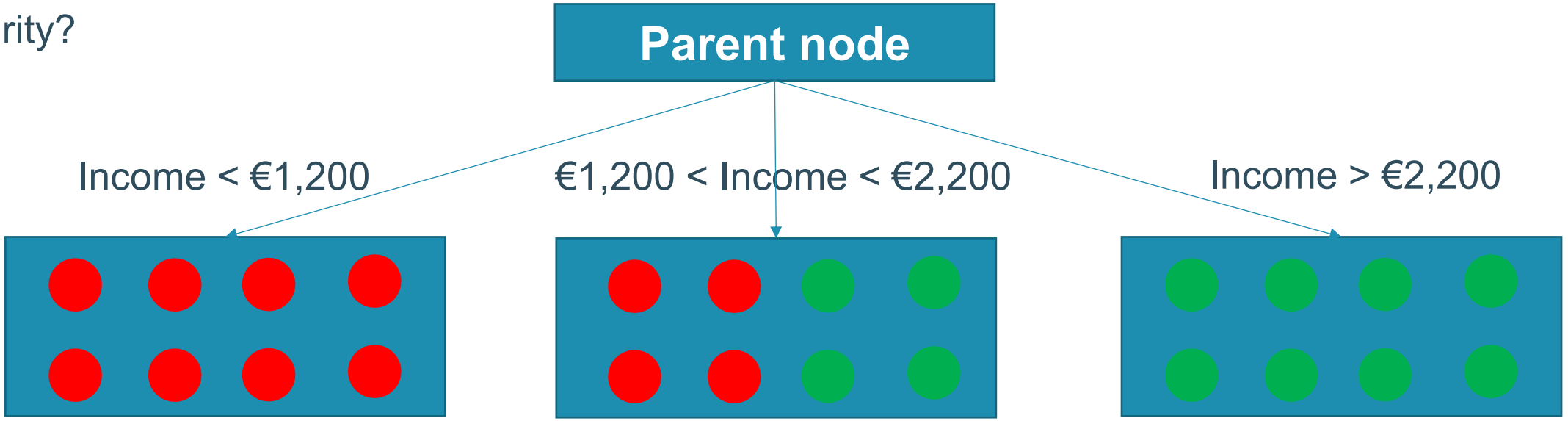
Splitting decision

- Two effects of a split are taken into account:

1. The reduction in ***impurity***
   - Do we improve the homogeneity of the resulting subsamples
   - Compared to the homogeneity of the initial sample that is split
   - How to **evaluate impurity**?
     - E.g.: entropy (C4.5), gini (CART), …
   - How to **compare impurity** of child nodes with impurity of parent node?
     - E.g.: Information Gain
2. The **number of observations** in the resulting subsamples

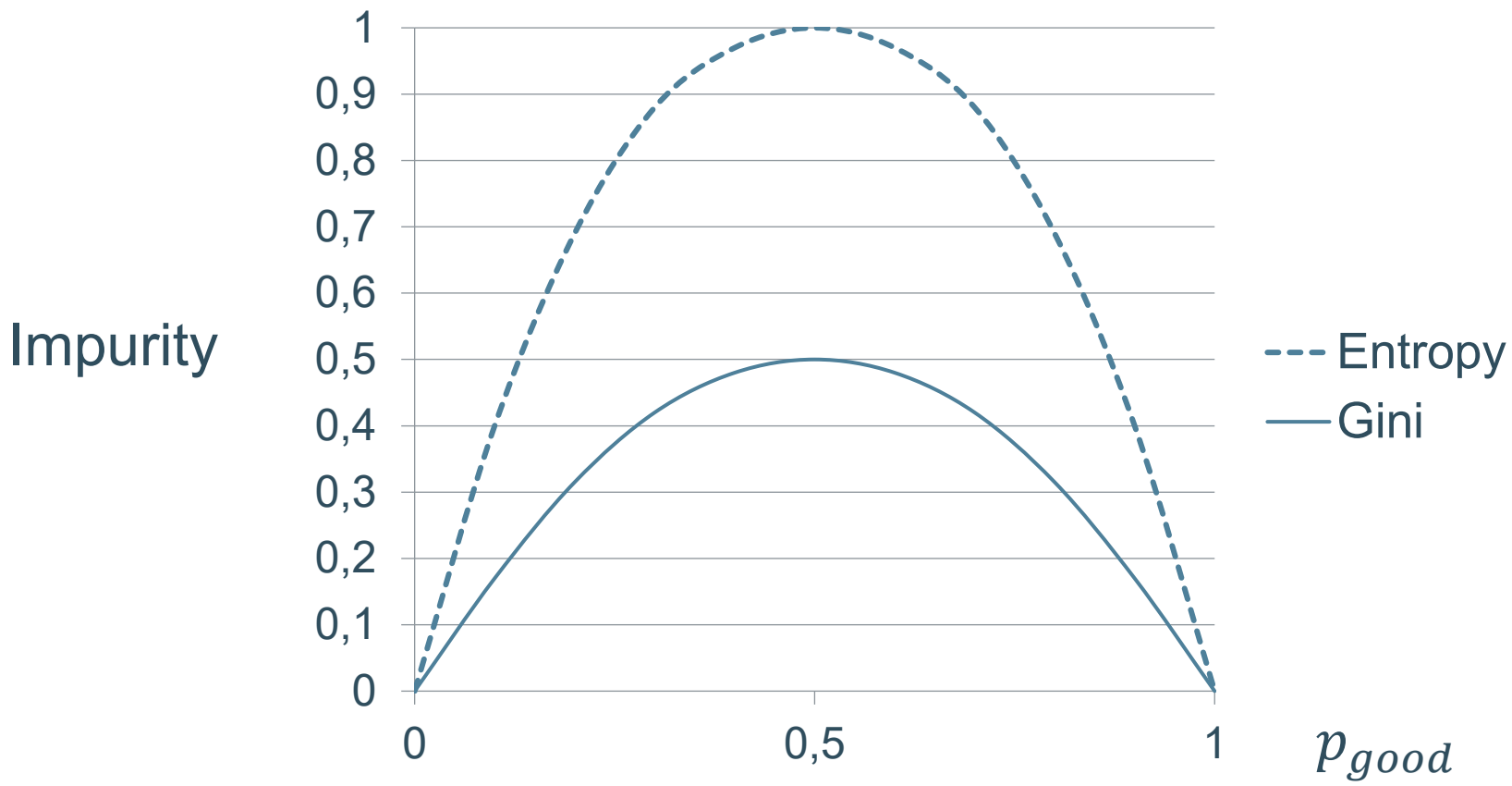
Splitting decision

- Impurity?



- **Entropy**: $-p_{red}\left(log_2(p_{red})\right) - p_{green}\left(log_2(p_{green})\right)$ ID3, C4.5
- **Gini**: $2p_{red}p_{green}$ CART

Splitting decision



Splitting decision

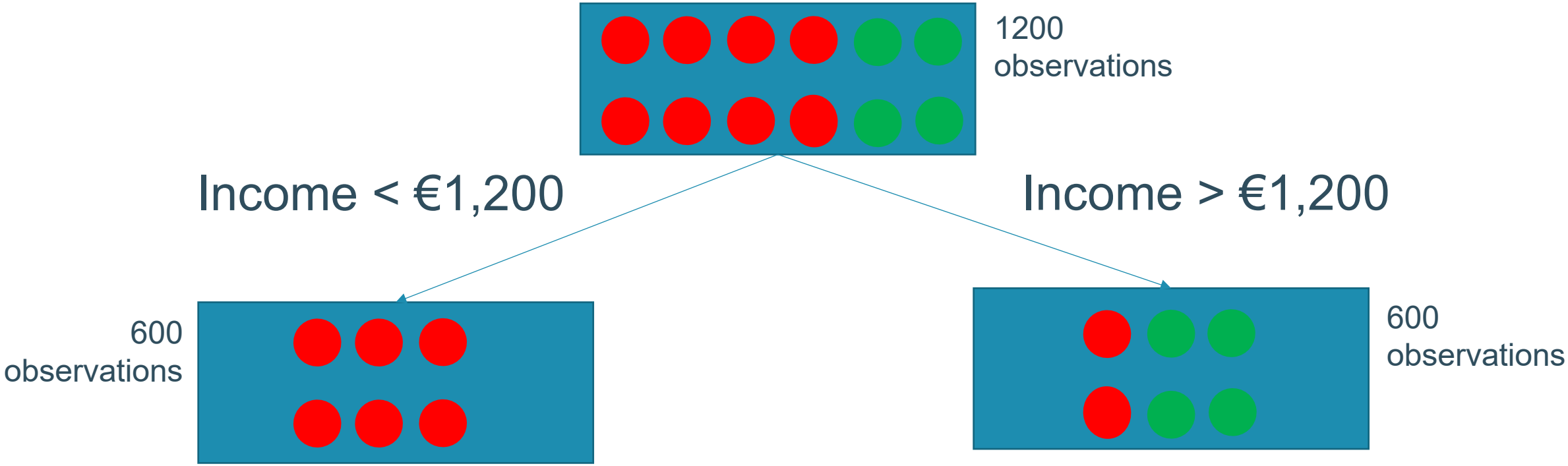
- **Problem**: how to compare the impurity of the resulting subsamples across all potential splits?
  - Account for the number of observations in the resulting subsamples!

- **Solution**: information gain measure

$$\boldsymbol{Information\ gain} = impurity_{parent} - \sum p_{child} \cdot impurity_{child}$$

  - p_{child}: proportion of observations of the *parent node* in a *child node*
  - Information gain = **weighted decrease in impurity**

Learning objective: *see* the underlying principles, i.e., the intuitive idea behind the workings of an algorithm, in the mathematical formulation



Gini parent node = $2 \cdot \frac{800}{1200} \cdot \frac{400}{1200} = 0{,}44$

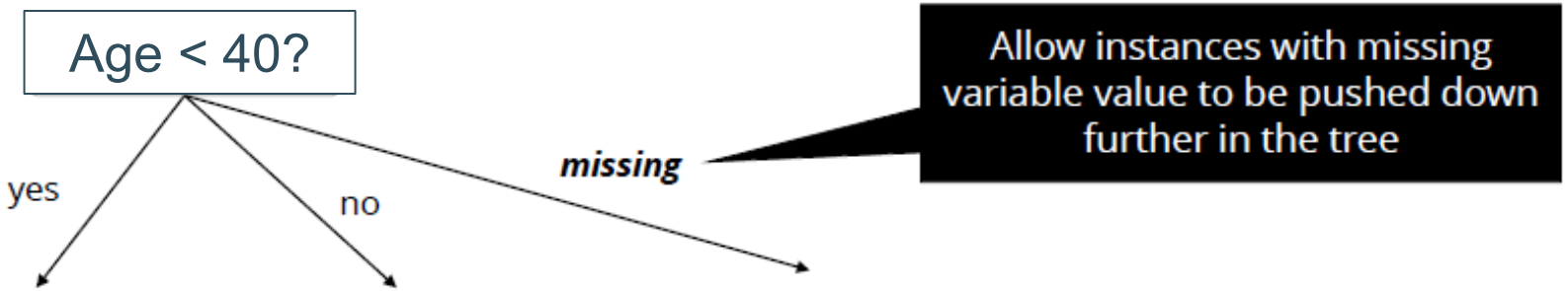
Gini left child node = 0

Gini right child node = $2 \cdot \frac{200}{600} \cdot \frac{400}{600} = 0{,}44$

Information gain = $0{,}44 - \frac{600}{1200} \cdot 0 - \frac{600}{1200} \cdot 0{,}44 = 0{,}22$

Splitting decision

- Possible splits that are evaluated:
  - Continuous variables: all possible values (<)
  - Categorical variables
    - Nominal: all possible combinations of values (=)
    - Ordinal: all possible values (<)
  - Missing values:
    - Can be added to the child node so as to maximize information gain
    - Separate group



Splitting decision

Cf. Chapter 2: WOE – binning variables
Trade-off in determining #bins between
- Predictive power (e.g., in terms of IV)
- Stability & interpretability

More than two child nodes (multi-way split)?

- Information gain can be extended to evaluate a split in more than two child nodes
- However: the more resulting groups, the larger the information gain (potentially - typically)
  - **Extreme case reasoning**:
    - If you split a parent node in as many child nodes as you have observations
    - Then each child node will be 100% pure
    - Then the information gain is maximal and equal to the impurity in the parent node
    - This is the 'best possible split' … or isn't it?
  - Cf. supra, infra & Chapter 4 (evaluation – single sample method): more resulting groups = larger complexity = more risk of overfitting = may hurt generalization power
- **So**: splitting criterion for multi-way splits should '***punish***' for more groups, to correctly **balance between increased complexity and increased information gain**
  - Detailed discussion on multi-way decision trees is beyond the scope of this course

Splitting decision

- Decision tree learning procedure: **recursive partitioning algorithms** (RPA)
  - In a node, all possible splits are evaluated
  - The best possible split is selected
  - Repeat

How long should we repeat?
→ Stopping decision! (cf. supra)

- Note:
  - We select splits iteratively – each time selecting the best **single** split
    - Such an approach is called a '**greedy' approach**
    - **Because you are greedy in each step of the procedure, taking as much as you can in terms of improving the result**
  - We do not select splits which, together with subsequent splits, result in a better segmentation
    - I.e., multivariate splits
    - There is, typically, no forward looking, *smarter* strategy adopted
  - Why?

Splitting decision

- Why don't we?
  - 'Globally optimal' tree learners have been proposed in the literature
    - Evaluating and selecting **combinations of multiple splits**
    - These are computationally more expensive: the search space dramatically grows!
  - The number of possible trees grows **exponentially** with:
    - The number of variables you have in the dataset
    - The number of values of these variables
    - The number of levels in the tree, combinations of splits, you want to evaluate at a time
  - The number of possible trees is/may be infinite
    - We cannot evaluate all possible trees to select the globally optimal tree
    - **Heuristic** approach needed → similar to **optimization approaches** which aim to find the global optimum, or at least a good solution, in a stepwise manner!

Cf. Chapter 3: variable selection

Stopping decision

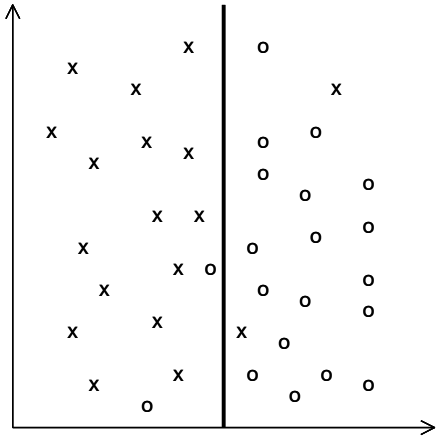
- How to decide when to stop growing the tree?
  - Why do we have to stop growing the tree before we reach minimum impurity?
- Possible approaches:
  - Information gain above a minimum threshold
  - Maximum size of the tree:
    - Number of ‘levels’
    - Number of leafs = number of rules = number of segments
  - Minimum number of observations per leaf
  - …
- None of these approaches is ‘optimal’ - but, what do we mean with optimal?
- What is our evaluation measure for assessing the quality of the resulting tree?

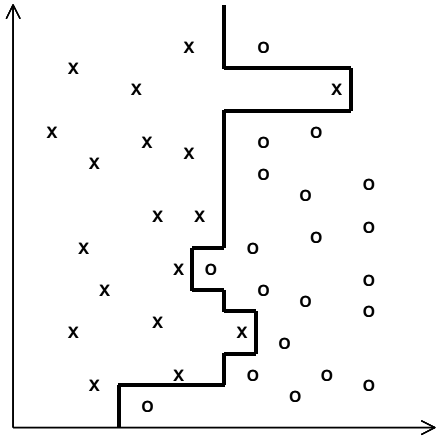
Stopping decision

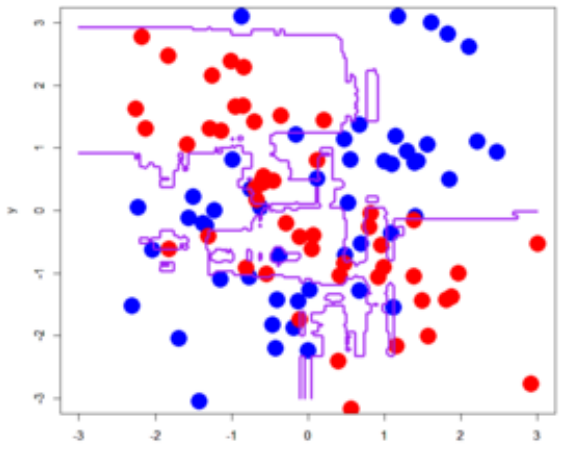
Complexity ~ overfitting

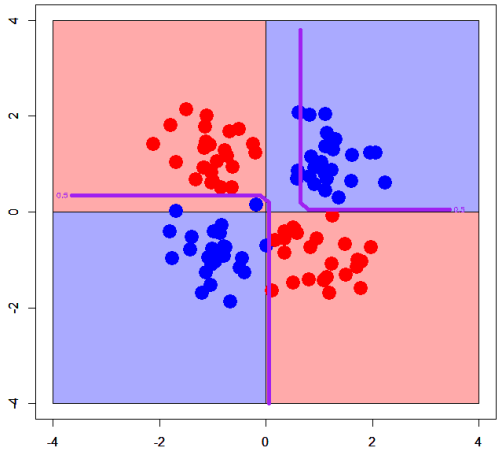
- The tree should ***generalize*** well
  - I.e., it should accurately predict new, unseen observations
  - A model learned from data, **is** that data, transformed (Y = f(X))
  - Therefore, a model will 'predict' well the data that was used to learn the model
  - Overfitting! Memorizing the data set! Fitting the noise!

Large tree resulting in a complex decision boundary = overfitted tree









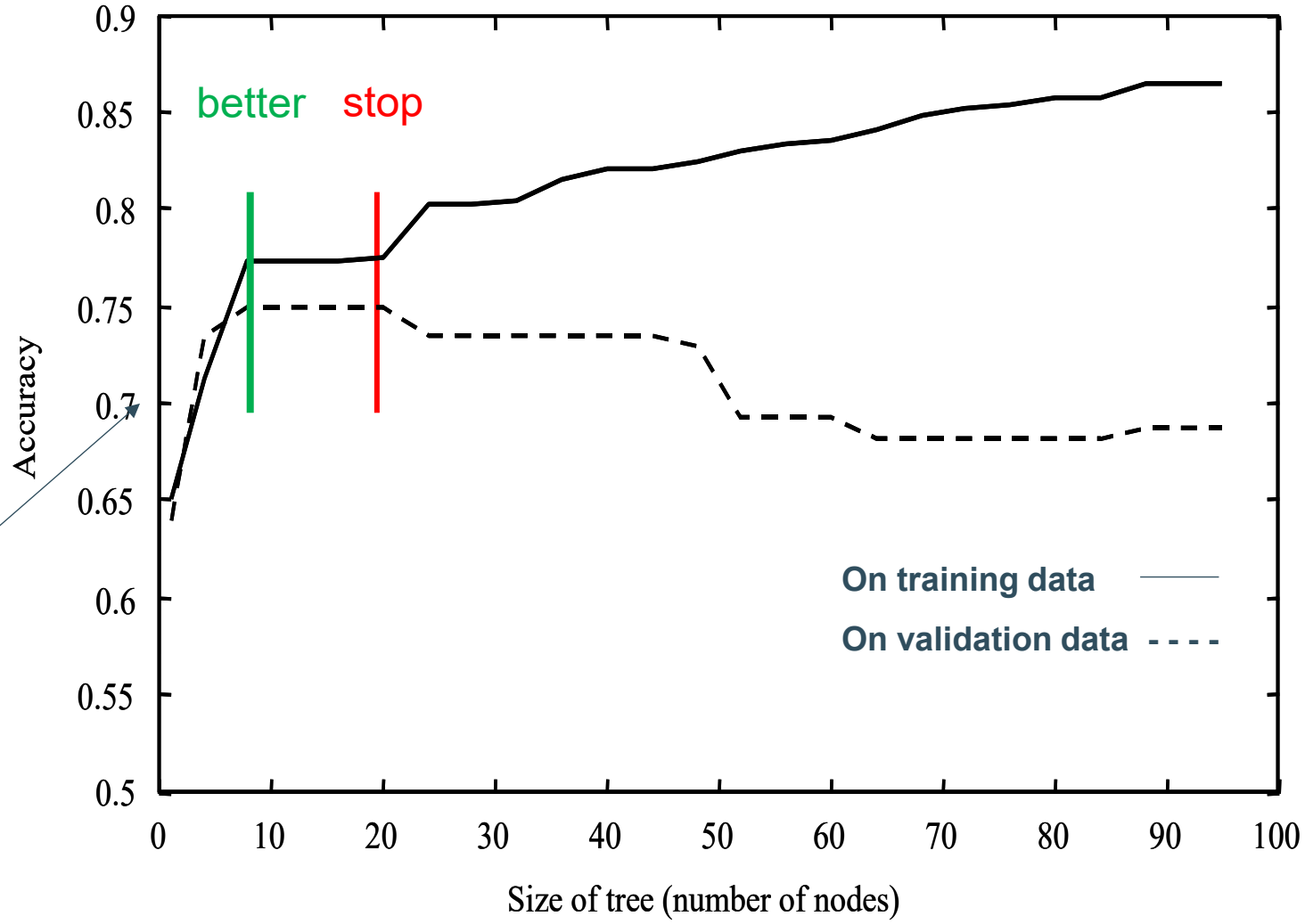
Stopping decision

- To decide when to stop growing the tree:
  - *Simulate* behavior on new data
  - Using a **validation set**: random subsample of the training data (typically: 30%)
  - Evaluation (cfr. Chapter 4):
    - **Misclassification error**: % of incorrectly predicted observations
    - **Classification accuracy**: % of correctly predicted observations
    - Also called PCC: percentage correctly classified

Stopping decision

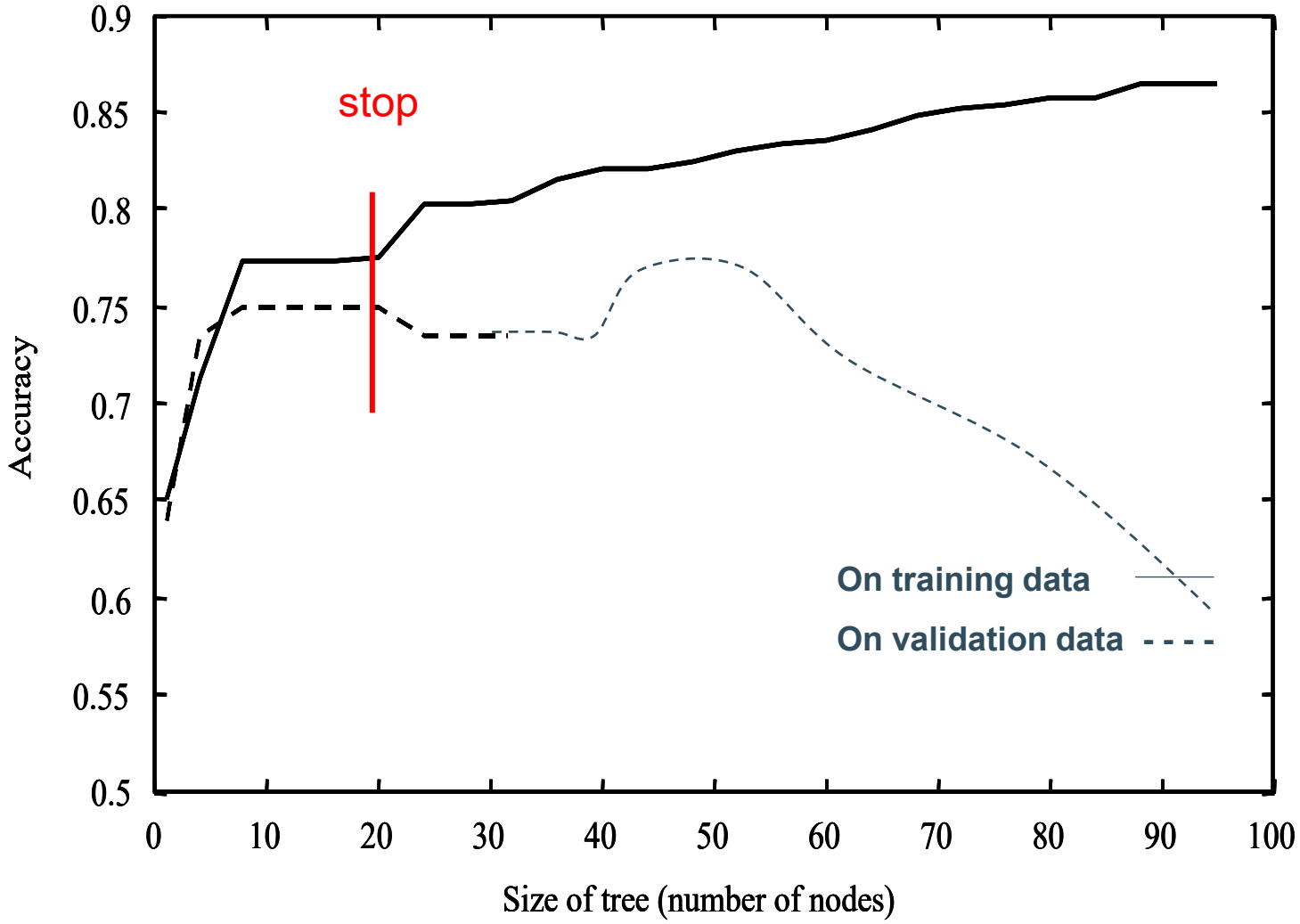
- Early stopping (pre-pruning)

In fact, this size of the tree is better: same accuracy but smaller size, less complex!



Stopping decision

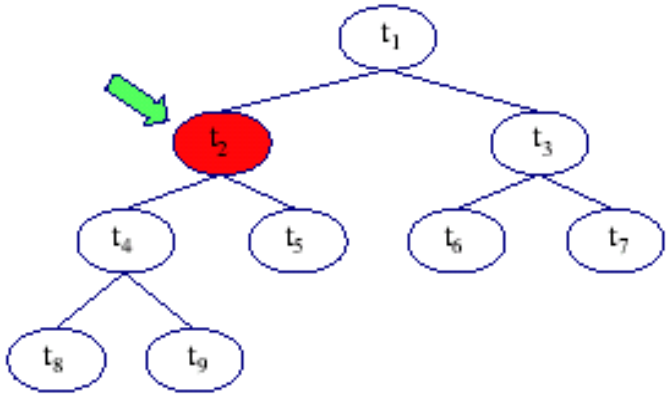
- Potential problem with early-stopping: **non-convex curve**
- Solution: post-pruning!



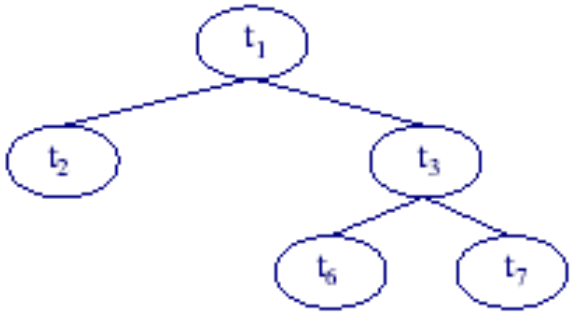
Stopping decision

- Post-pruning:
  - Grow *full* tree, with accuracy on the training set = 100%
  - Cut branches so as to optimize performance on validation set

Pruning Tree T in node t_2

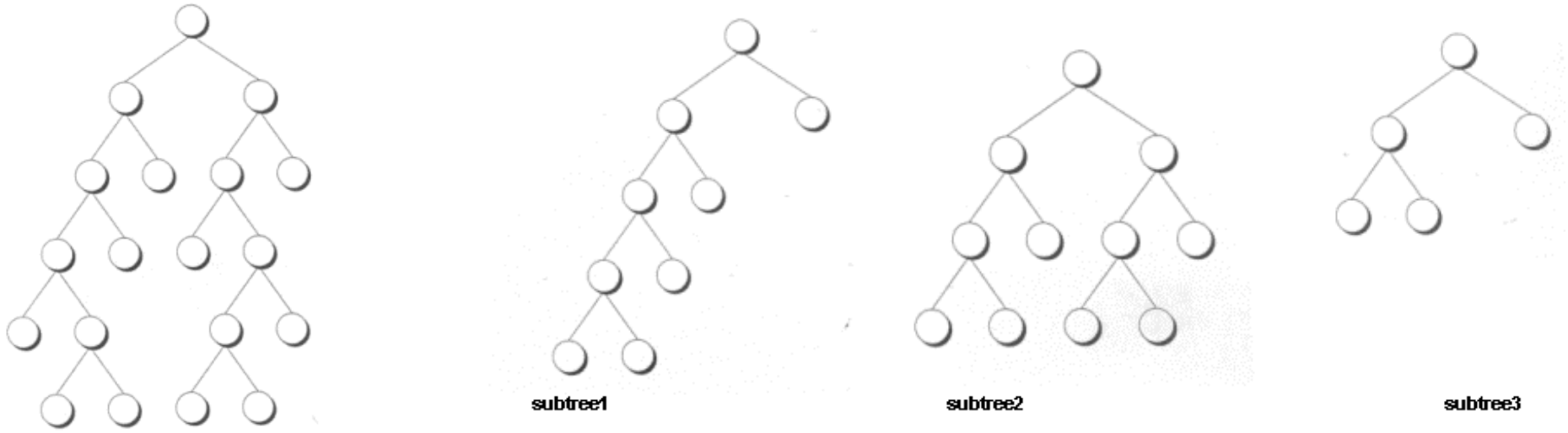


T after pruning in t_2



Stopping decision

- Various possibilities to prune:
  - Optimization problem
  - **Exhaustively** evaluate performance of all subtrees (limited number - search space)

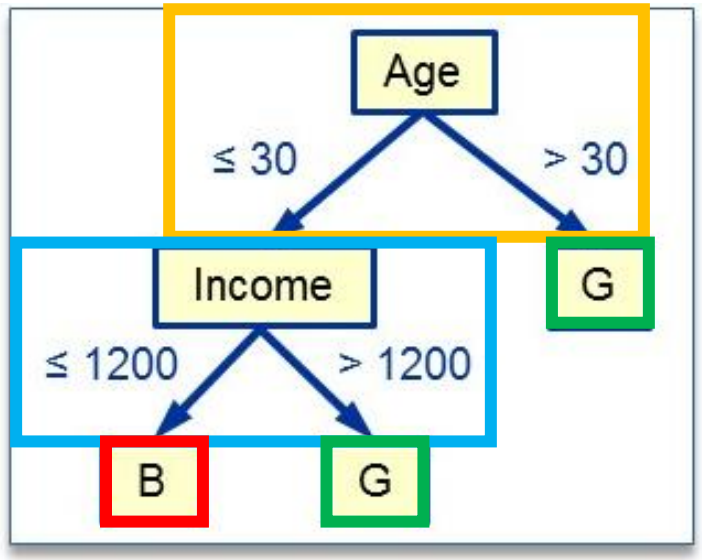


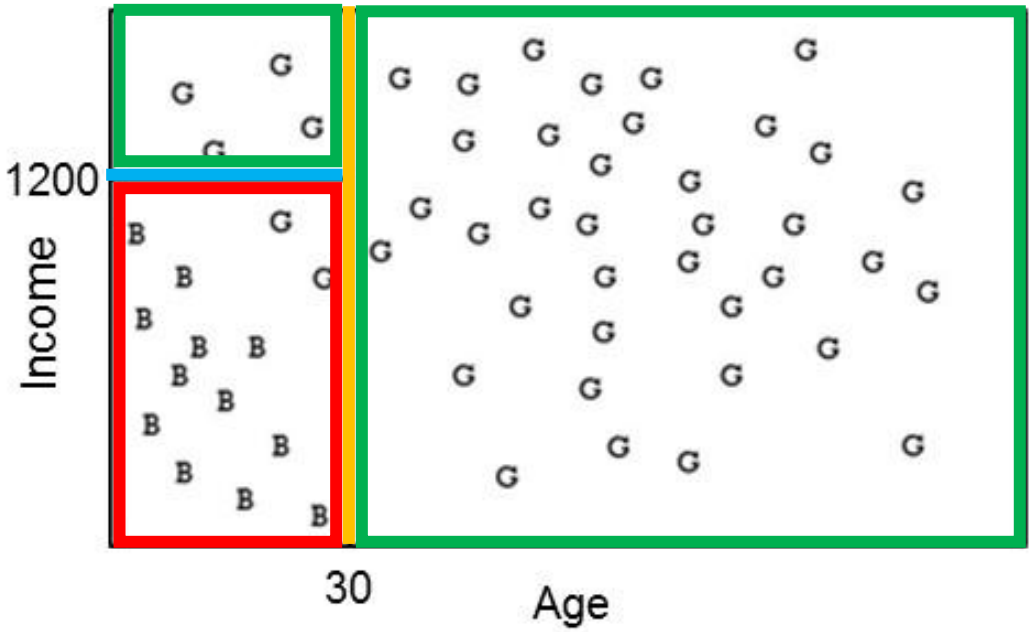
Tuning

- Validation set for deciding on the size of the decision tree:
  - = tuning the decision tree size
  - Often used in data science: **tuning *hyperparameters***!
  - See later chapters:
    - Number of neurons in an artificial neural network
    - Number of trees in an ensemble
    - …
- Terminology:
  - Validation set = tuning set
  - Training set = development set

Decision boundary

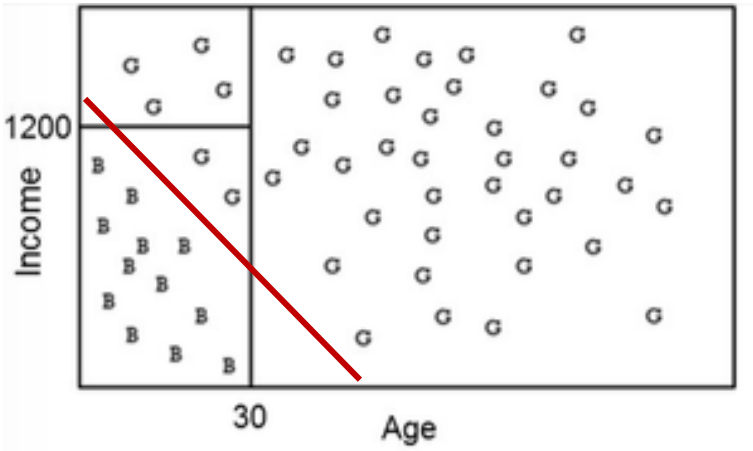
- How does the decision boundary of a decision tree look like?





Decision boundary

- **Interaction effects**!
  - When the relation between predictor A (e.g., income) and the target variable depends on the value of another predictor B (e.g., age)
  - Then we have an interaction between predictors A and B
  - Meaning: for different subsamples, there are other variables which explain the outcome in terms of the target variable
    - *Income is predictive for good/bad, at least, for age < 30*
  - Cf. CHAID (Hartigan 1975): **CH**i-squared **A**utomatic **I**nteraction **D**etection
- **Note**:
  - A decision tree **cannot learn and incorporate linear relations**!
  - Linear & logistic regression can (cf. Chapter 6), but then again cannot incorporate interaction effects



Decision boundary

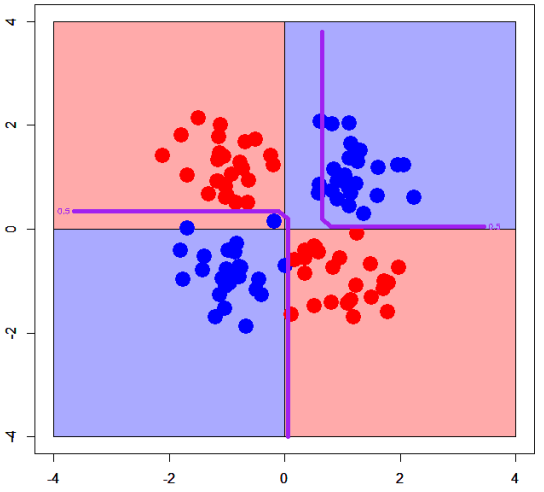
- Given the ***specific abilities*** of decision trees:
  - Decision trees will sometimes perform well, sometimes they will not
  - Depending on ***the type of problem***, i.e., the **type of relations or patterns** in the data, decision trees will or will not be able to accurately classify:
    - (X-)OR-style interactions: **YES**
    - Linear relations: **NO**

XOR

X1	**1**	1	0
	0	0	1
		0	**1**
		X2	

OR

X1	**1**	1	1
	0	0	1
		0	**1**
		X2	



Decision trees: advantages and disadvantages

- **Advantages**
  - Interpretable: rules
  - Non-parametric: no assumptions regarding the distribution of the sample
  - Robust with respect to input data
    - Missing values
    - Outliers
    - Variable selection
    - Categorical and continuous variables
- **Disadvantages**
  - Sensitive to changes in the training data: unstable classifier
    - Different split up in training and validation set yields possibly a different tree: derived relations unstable?
  - Sensitive to imbalanced class distribution
  - Predictive power ☹: weak classifier

→ **Certainly with imbalanced data!!**
→ Cf. Chapter 10 / Part 2: sampling procedures

Uses of decision trees

- Predictive model: careful, depending on the setting (requirements)
- Data exploration: no preprocessing required
- Variable selection in preprocessing step using information gain
- Segmentation prior to development of final predictive models
  - Segments are found based on interactions, so meaningful segmentation
  - Logit leaf model: a logistic regression model is fitted in the leafs
- Coarse classification (multi-way decision trees)
- Ensembles! (cf. Chapter 5)

De Caigny, A., Coussement, K., & De Bock, K. W. (2018). A new hybrid classification algorithm for customer churn prediction based on logistic regression and decision trees. European Journal of Operational Research, 269(2), 760-772.

Case: customer churn

- Cellular phone churn data set
  - 10 predictors
  - Target variable: leave

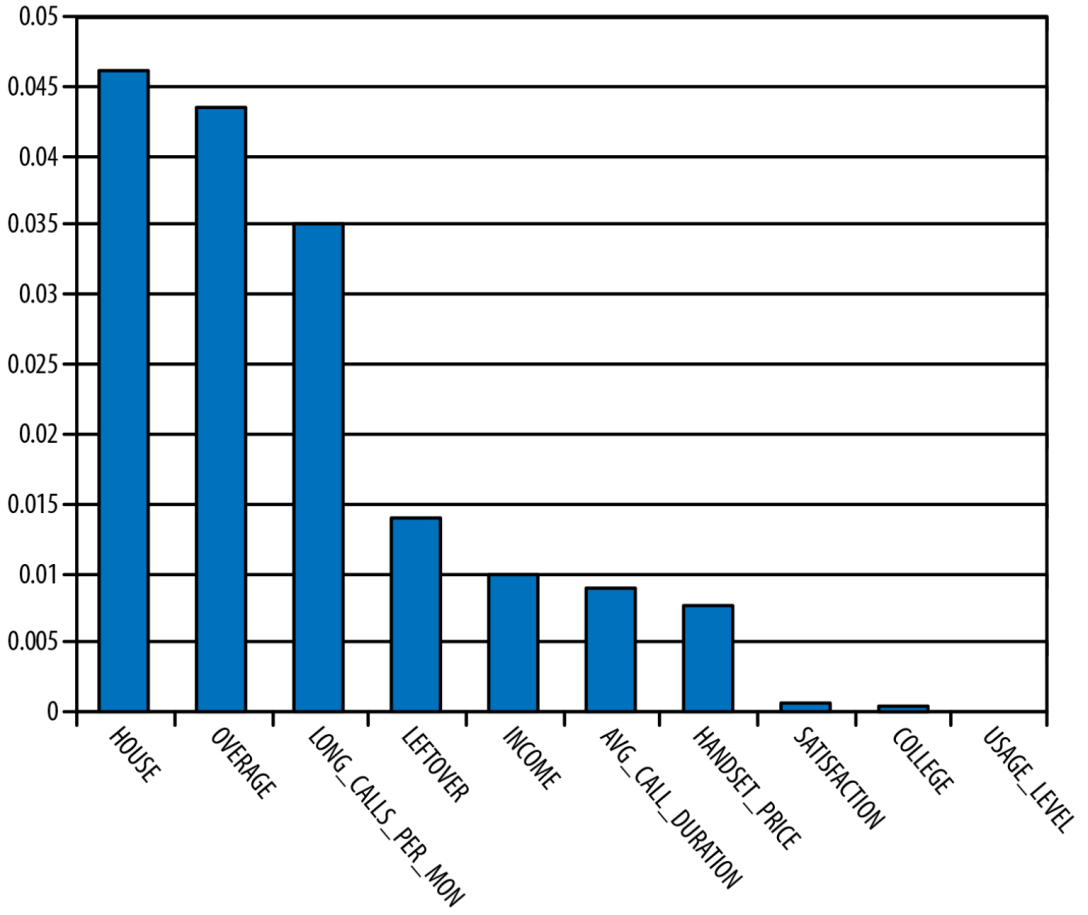
- Demographics
- Usage information

Variable	Explanation
COLLEGE	Is the customer college educated?
INCOME	Annual income
OVERAGE	Average overcharges per month
LEFTOVER	Average number of leftover minutes per month
HOUSE	Estimated value of dwelling (from census tract)
HANDSET_PRICE	Cost of phone
LONG_CALLS_PER_MONTH	Average number of long calls (15 mins or over) per month
AVERAGE_CALL_DURATION	Average duration of a call
REPORTED_SATISFACTION	Reported level of satisfaction
REPORTED_USAGE_LEVEL	Self-reported usage level
LEAVE *(Target variable)*	Did the customer stay or leave (churn)?

Case: customer churn

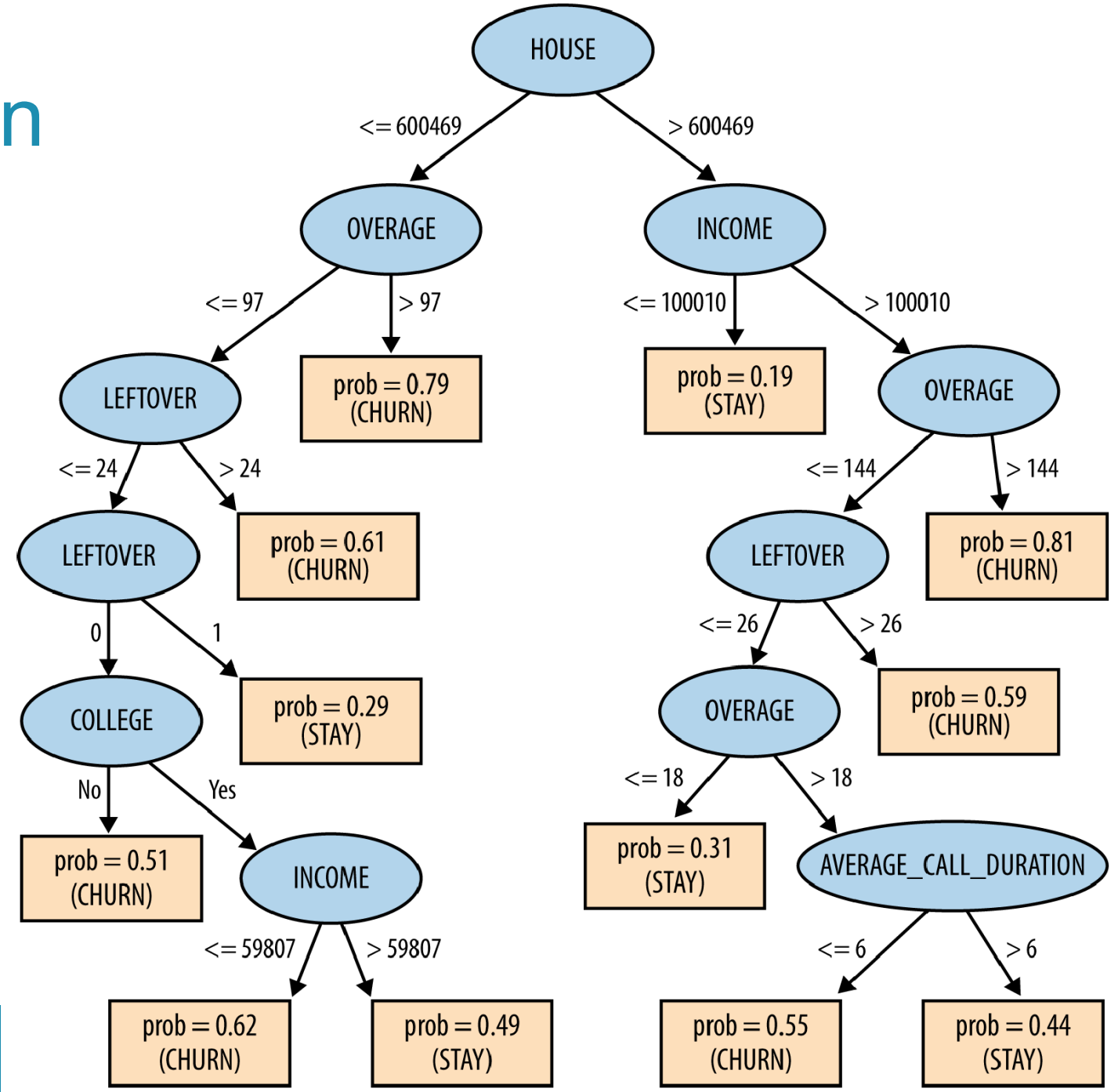
- Information Gain:

Rank	Info. gain	Attribute name
1	0.0461	HOUSE
2	0.0436	OVERAGE
3	0.0350	LONG_CALLS_PER_MON
4	0.0136	LEFTOVER
5	0.0101	INCOME
6	0.0089	AVG_CALL_DURATION
7	0.0076	HANDSET_PRICE
8	0.0003	SATISFACTION
9	0.000	COLLEGE
10	0.000	USAGE_LEVEL



Case: customer churn

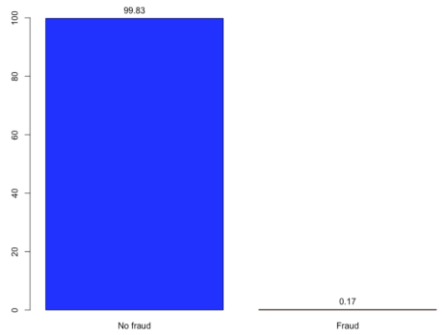
- Which customers would you target first?
- What is the decision rule to identify these customers?
- Can you interpret this rule?
- Can you interpret the entire tree?



2. Re-sampling

Re-sampling

- Typical issue in classification: imbalanced or *skewed* class distribution
  - E.g.:
    - Churn rate: 1% to 5%
    - Fraud rate << 1%
  - May lead to poor predictive performance



- Solution?
  - Classifiers ***may*** learn better from a dataset with a more balanced class distribution
  - **Re-sampling** to this end ***may*** improve predictive performance
    - Undersampling
    - Oversampling
    - SMOTE

Re-sampling

- **Undersampling**
  - Randomly removing observations from the majority class
  - Trial and error to determine optimal **undersampling rate** and resulting distribution
  - Test set remains untouched!



Re-sampling

Original data

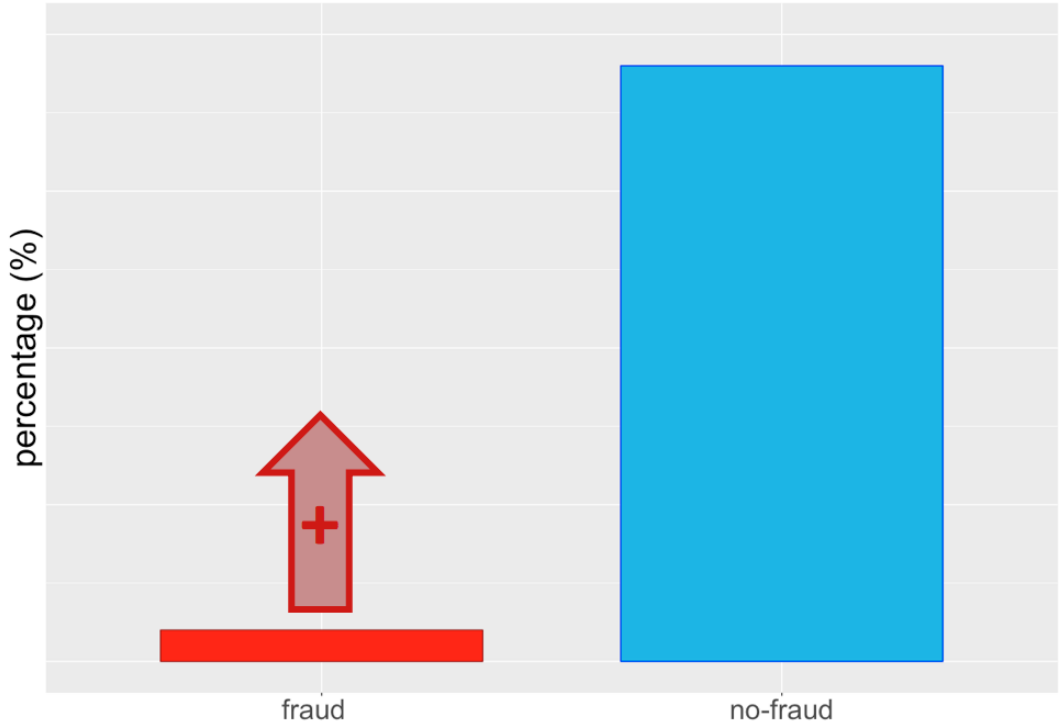
	ID	Variables	Class
Train	1		Fraudster
	2		Non-Fraudster
	3		Non-Fraudster
	4		Fraudster
	5		Non-Fraudster
	6		Non-Fraudster
Test	7		Non-Fraudster
	8		Non-Fraudster
	9		Fraudster
	10		Non-Fraudster

Undersampled data

	ID	Variables	Class
Train	1		Fraudster
	3		Non-Fraudster
	4		Fraudster
	6		Non-Fraudster
Test	7		Non-Fraudster
	8		Non-Fraudster
	9		Fraudster
	10		Non-Fraudster

Re-sampling

- **Oversampling**
  - Copying observations from the minority class
  - Trial and error to determine optimal **oversampling rate** and resulting distribution
  - Test set remains untouched!



Re-sampling

Original data

	ID	Variables	Class
Train	1		Fraudster
	2		Non-Fraudster
	3		Non-Fraudster
	4		Fraudster
	5		Non-Fraudster
	6		Non-Fraudster
Test	7		Non-Fraudster
	8		Non-Fraudster
	9		Fraudster
	10		Non-Fraudster

Oversampled data

	ID	Variables	Class
Train	1		Fraudster
	1		Fraudster
	2		Non-Fraudster
	3		Non-Fraudster
	4		Fraudster
	4		Fraudster
	5		Non-Fraudster
	6		Non-Fraudster
Test	7		Non-Fraudster
	8		Non-Fraudster
	9		Fraudster
	10		Non-Fraudster

Re-sampling

- **Over- and undersampling**
  - Copying observations from the minority class & removing observations from the majority class
  - Trial and error to determine optimal over- and undersampling rate and resulting distribution
  - Test set remains untouched!



Re-sampling

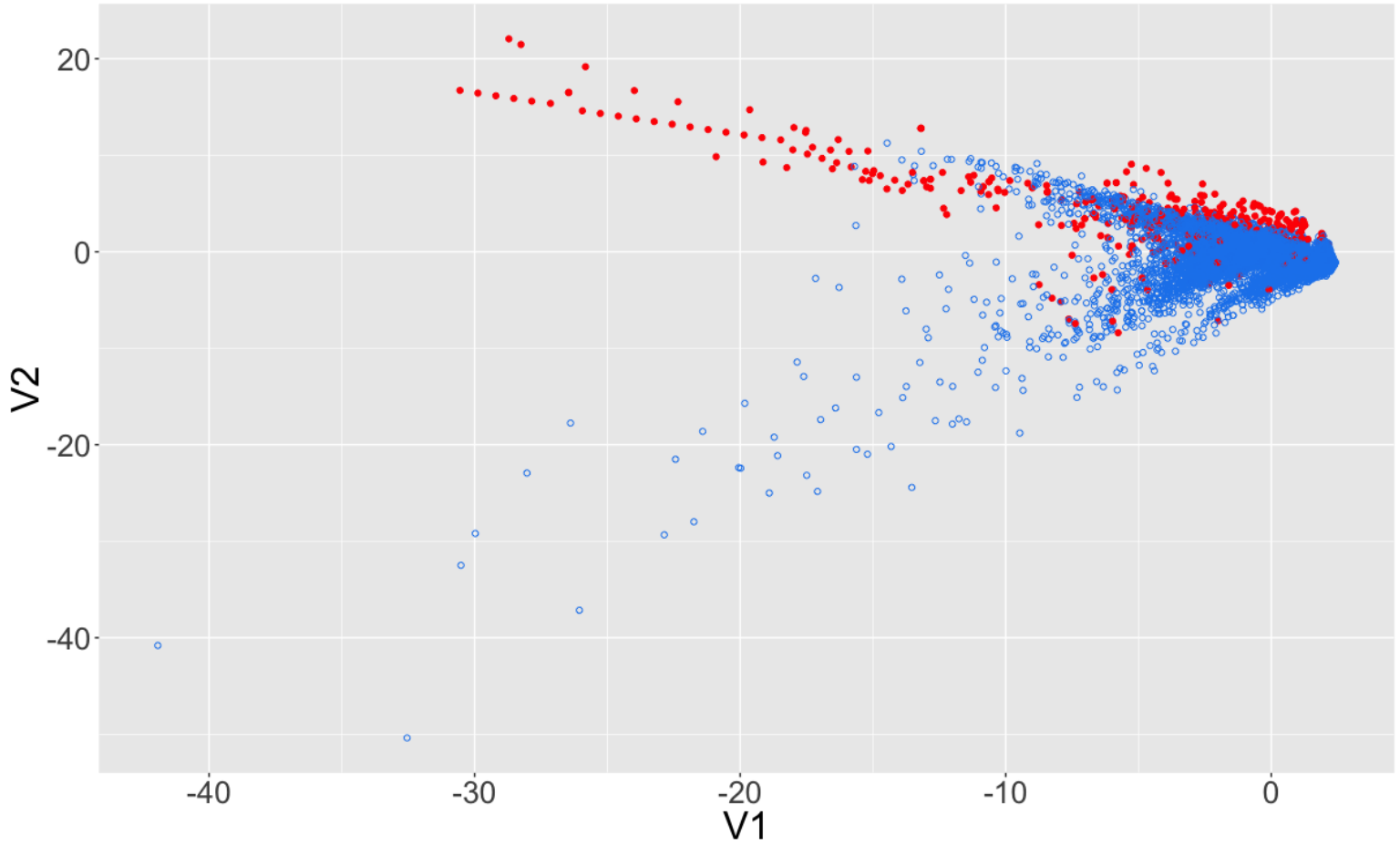
- What is **optimal re-sampled class distribution**?
- Trial and error:
  - Start from original data (e.g., 99%/1% non-fraud/fraud), build classifier and measure AUC on validation set
  - Use over-or undersampling to change class distribution, e.g., subsequently to 98%/2%, 97%/3%, …
  - Continue until AUC stagnates or drops
- Note: 80%/20% often adopted in industry
- Undersampling usually better than oversampling

Re-sampling

- Credit Card Fraud Detection dataset on Kaggle
  - https://www.kaggle.com/mlg-ulb/creditcardfraud
  - ~ 300K anonymized credit card transactions labeled as fraudulent or legitimate
- About the data...
  - Numerical variables: V1, V2, ... , V28
  - Time = seconds elapsed between each transfer and first transfer
  - Transaction amount
  - Class = value 1 in case of fraud and 0 otherwise

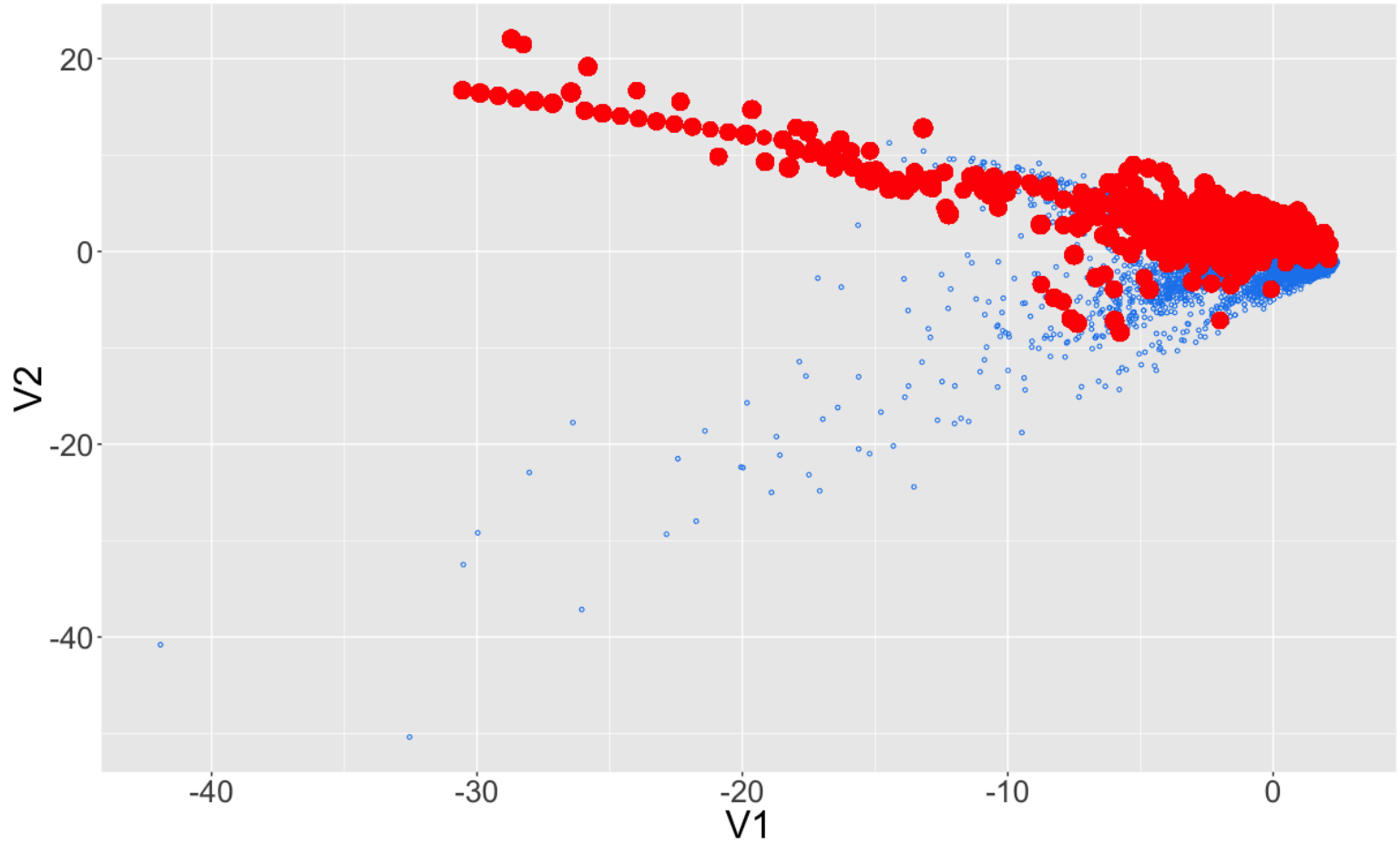
Re-sampling

2% fraud cases (in red)!



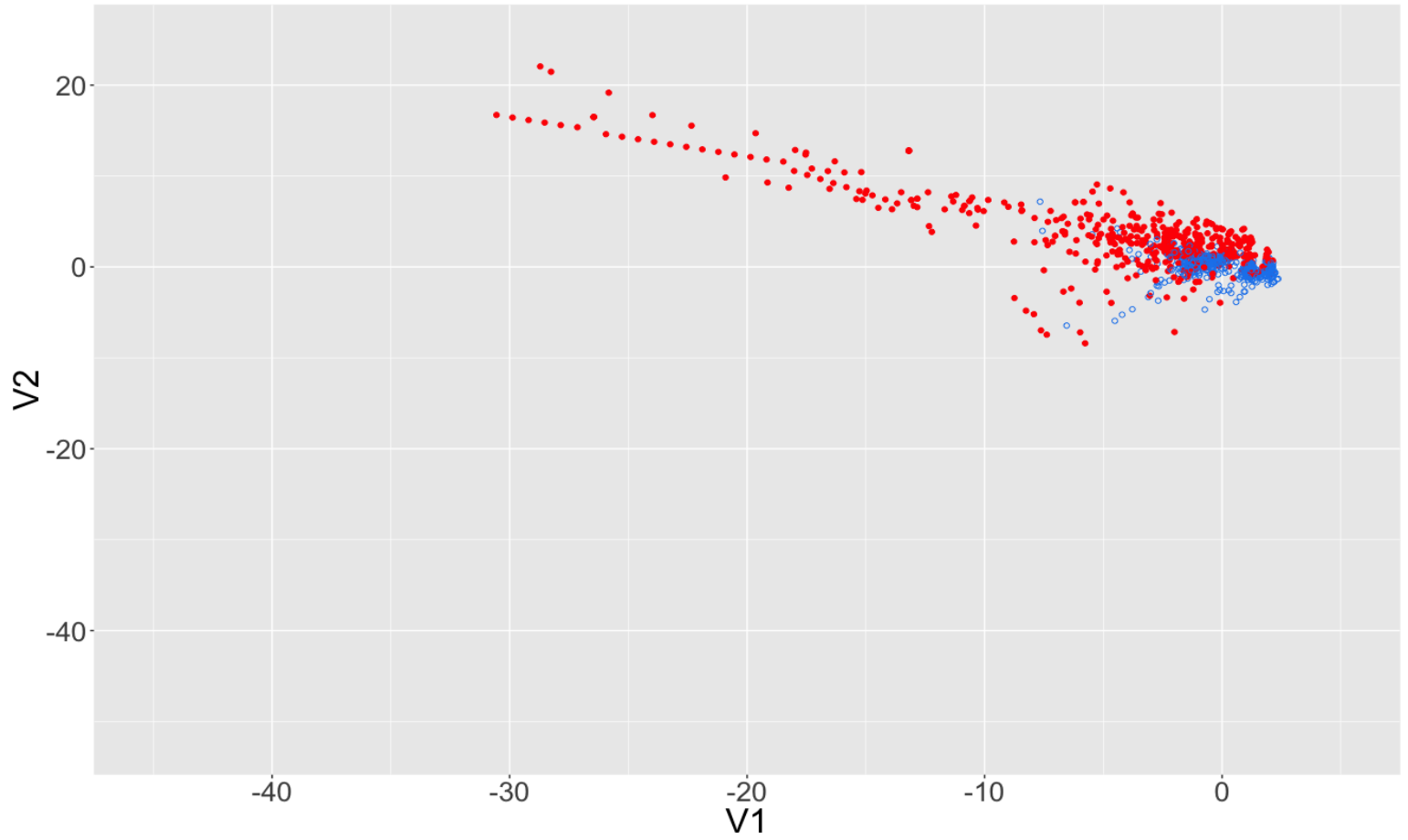
Re-sampling

Oversampling: 50/50
(size of dot corresponds to frequency)



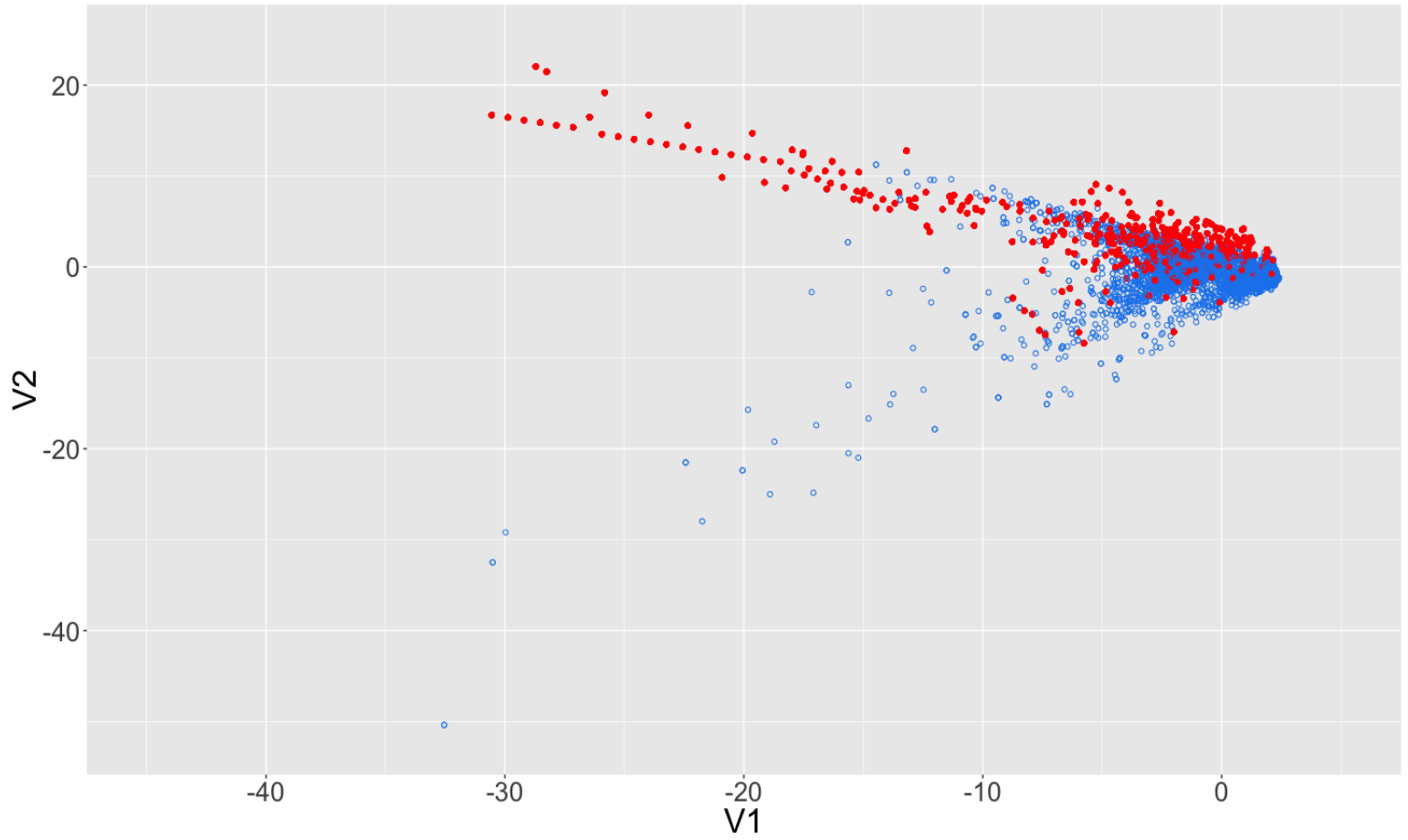
Re-sampling

Undersampling: 50/50



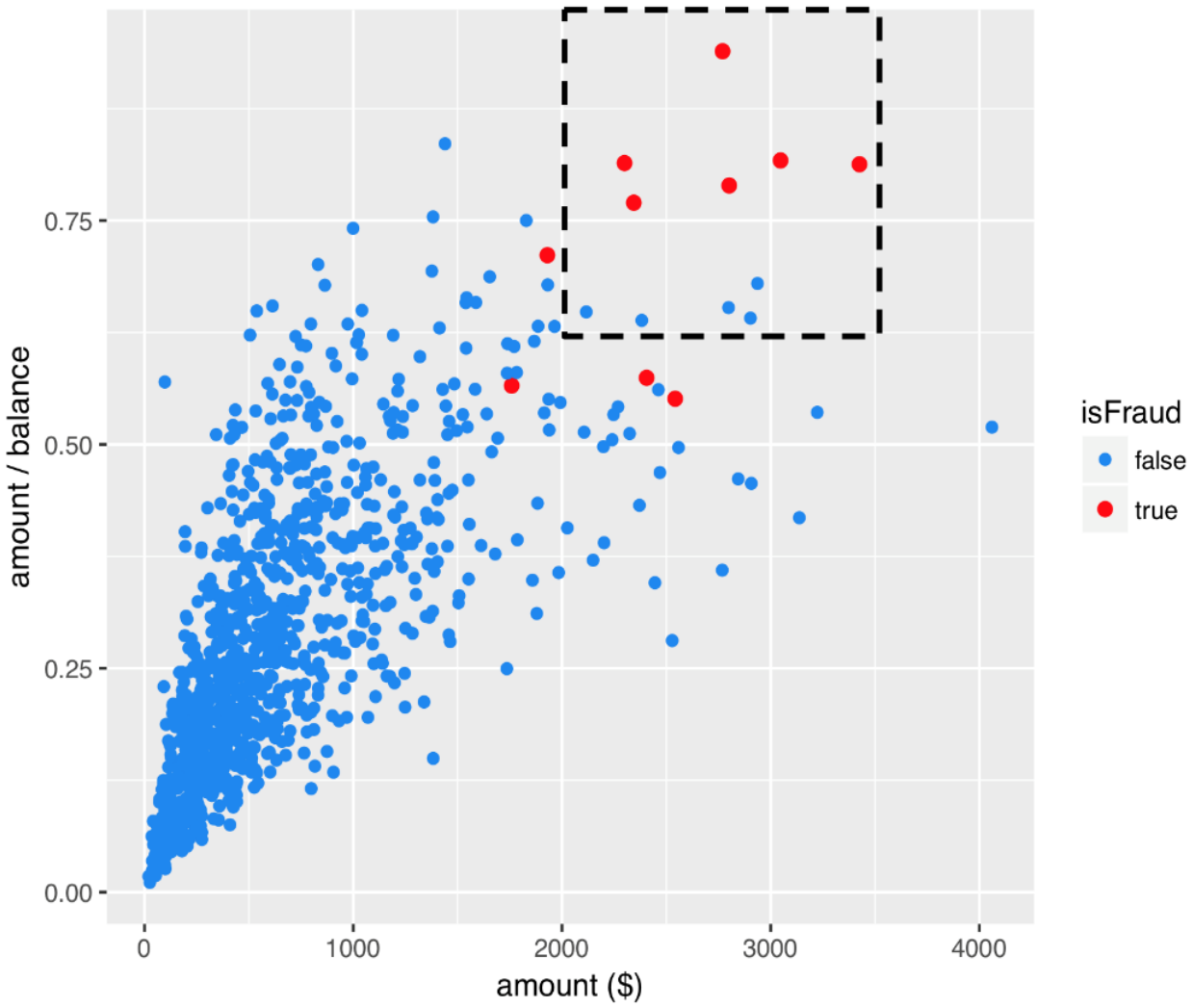
Re-sampling

Oversampling + Undersampling



Re-sampling

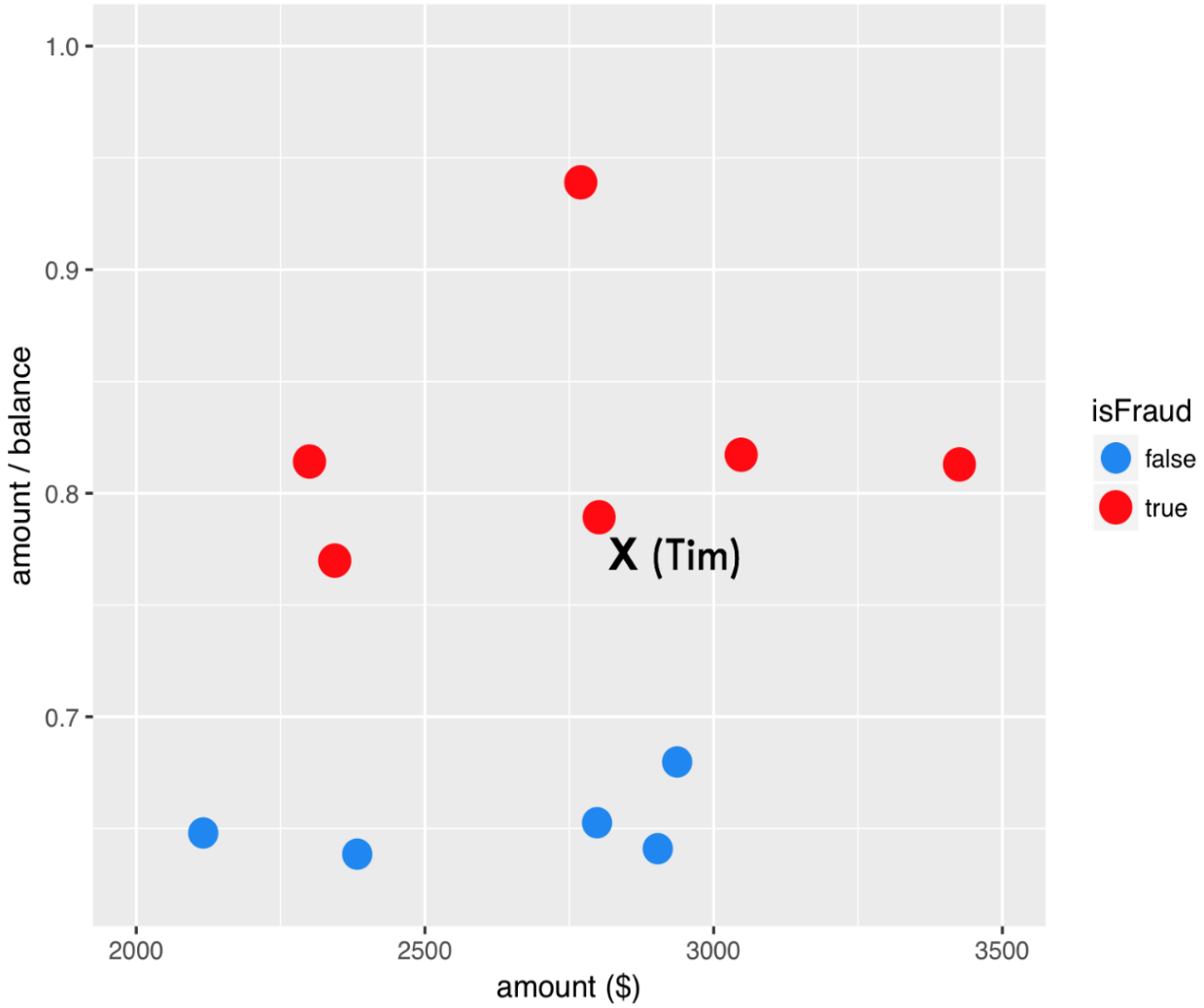
- Synthetic Minority Oversampling technique (SMOTE) (Chawla et al., 2002)
  - Oversample minority class by **creating synthetic examples**
  - Can be combined with undersampling majority class



Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, *16*, 321-357.c

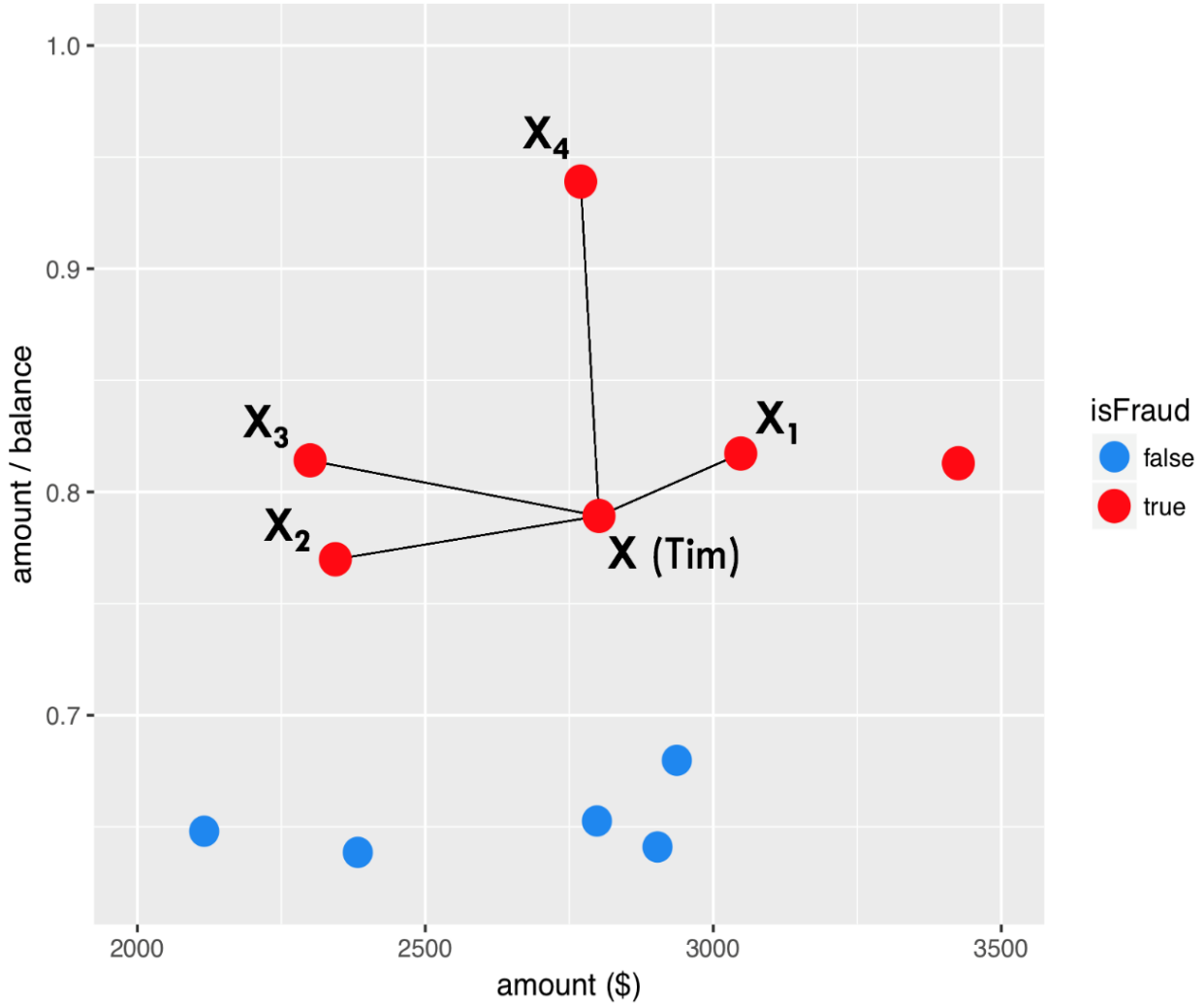
Re-sampling

- **Step 1:** For each minority class observation, determine ***k nearest neighbors***



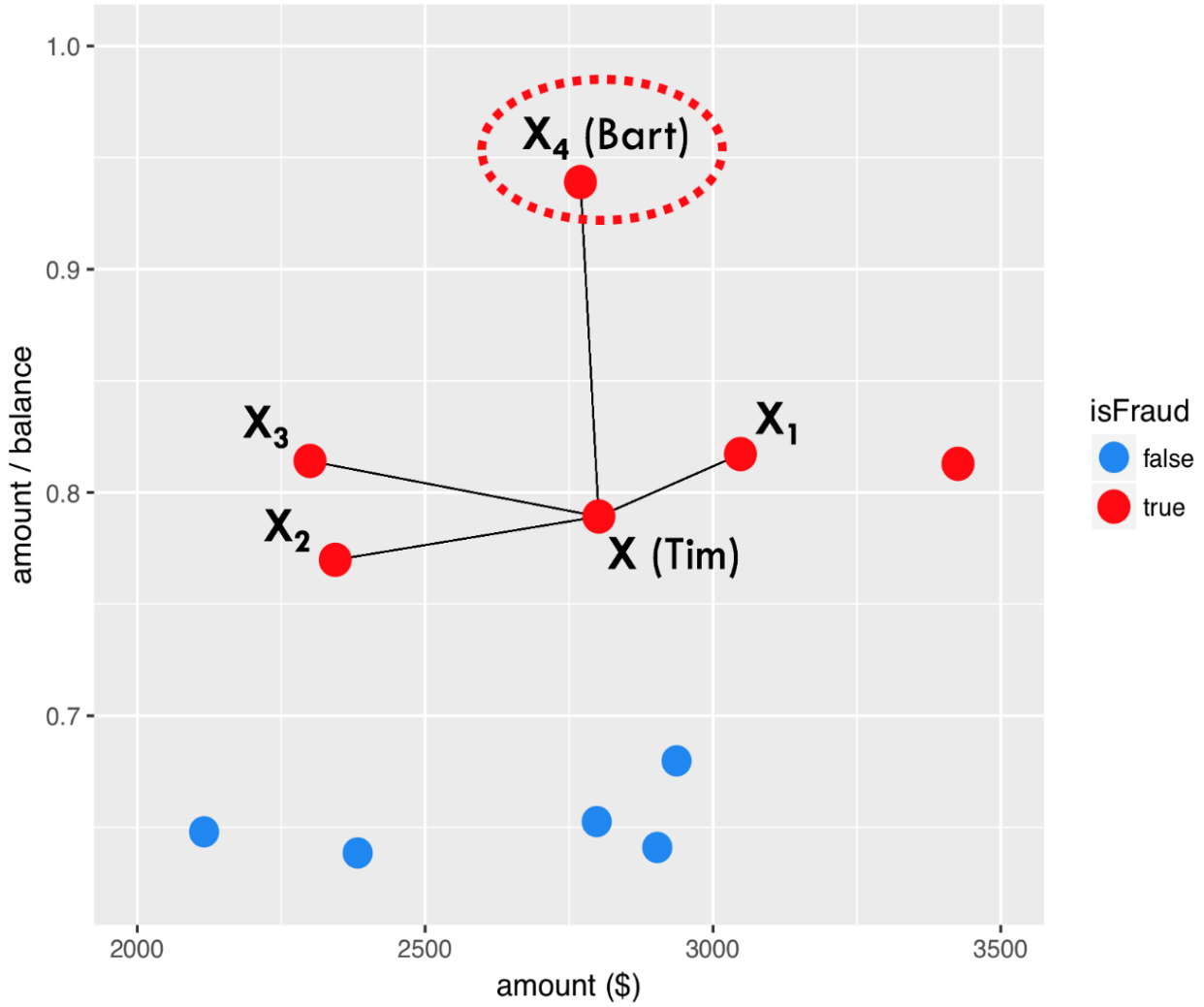
Re-sampling

- **Step 1:** For each minority class observation, determine ***k nearest neighbors***



Re-sampling

- **Step 1:** For each minority class observation, determine ***k nearest neighbors*** and randomly select one of them
- **Step 2:** Generate synthetic obs.:
  - Multiply difference for all variables with a random number between 0 and 1
  - Add this difference to the minority class observation



Re-sampling

X (Tim)

Amount	Ratio
2800	0.79

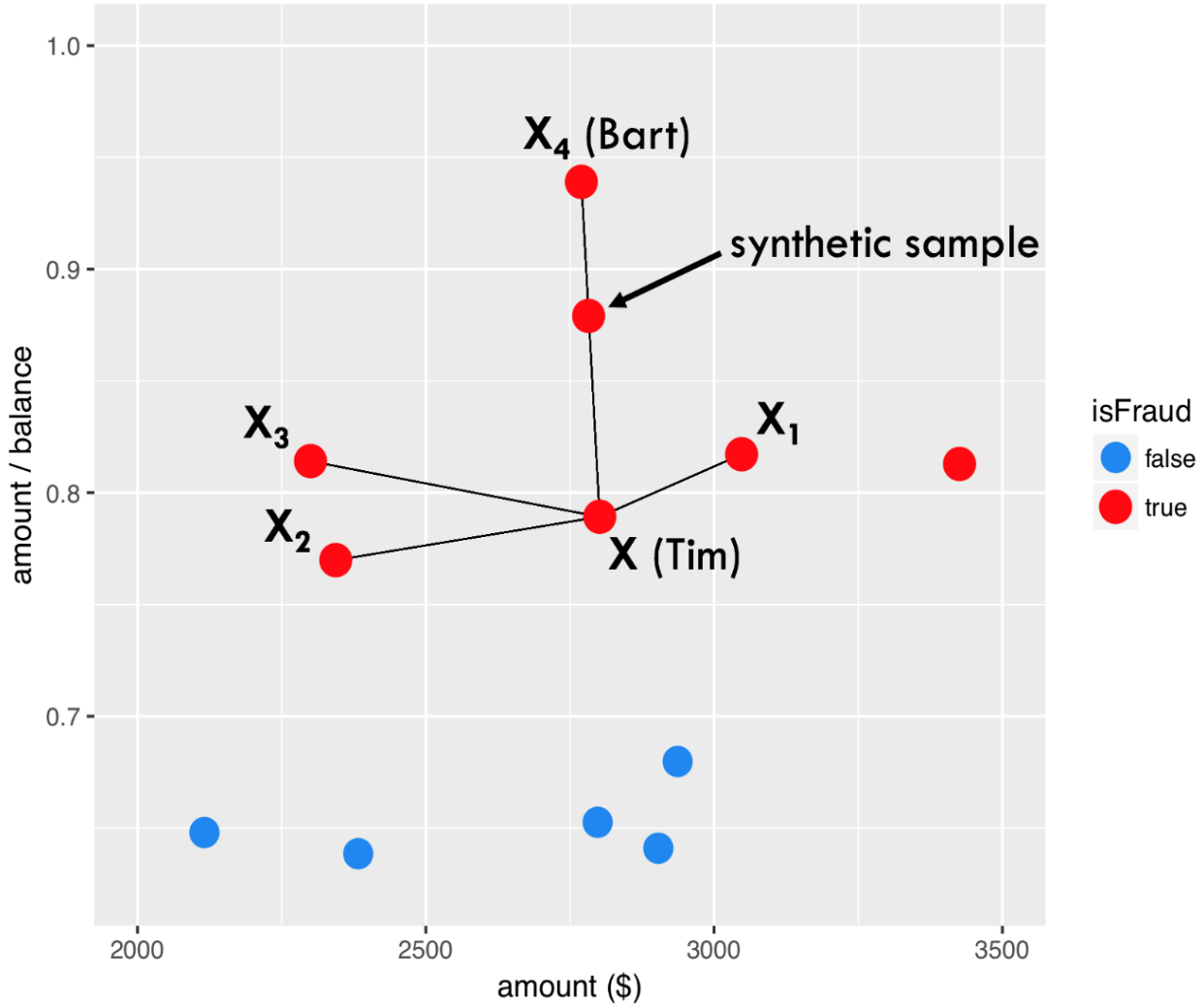
X_4 (Bart)

Amount	Ratio
2770	0.94

Choose random number between 0 and 1, e.g. **0.6**

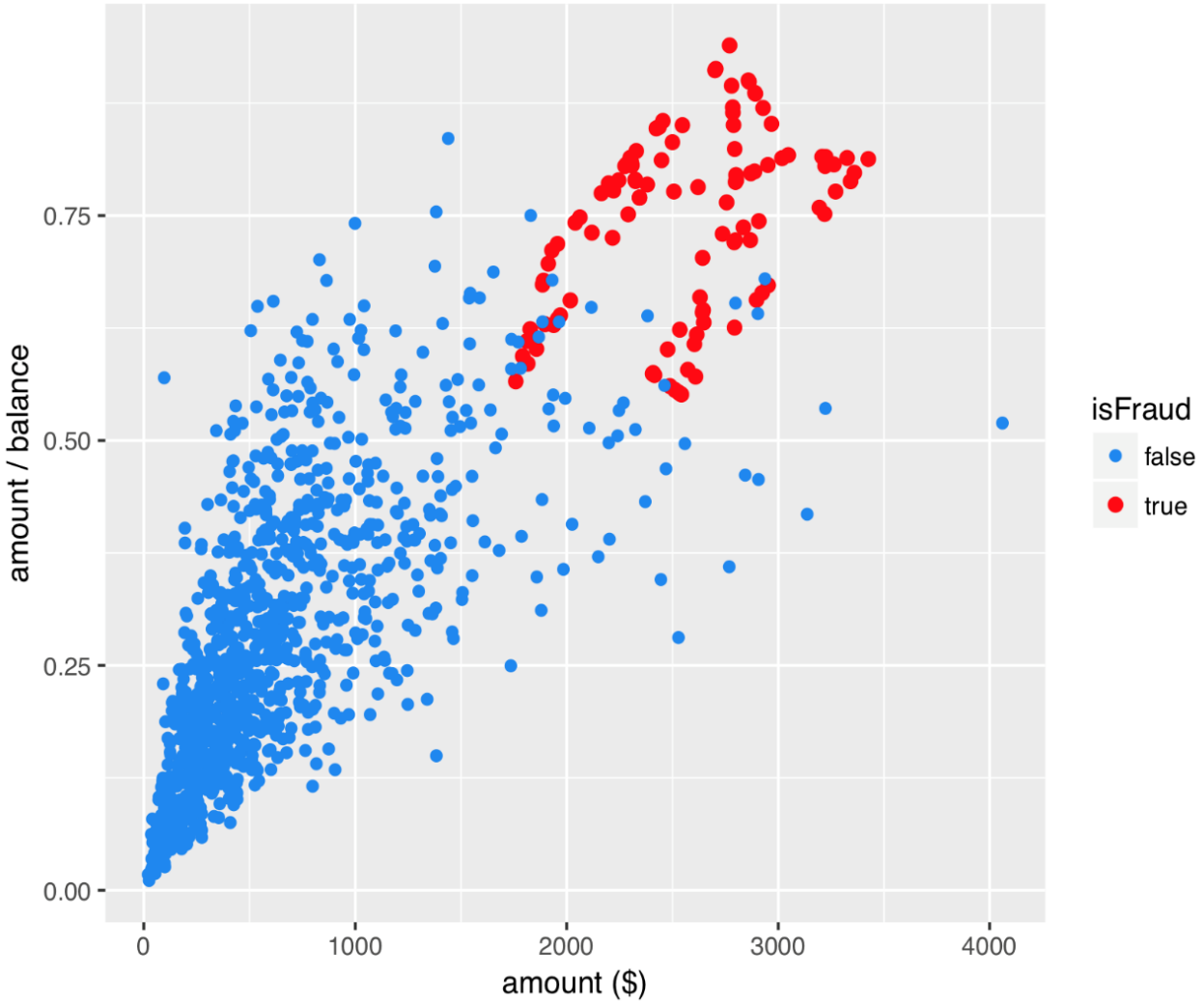
Synthetic sample

Amount	Ratio
2800 + **0.6** * (2770 - 2800) = **2782**	0.79 + **0.6** * (0.94 – 0.79) = **0.88**



Re-sampling

- Resulting dataset after applying SMOTE
  - Original dataset contained 100 fraud cases
  - SMOTE created 10 extra synthetic fraud observations
  - Resulting in 110 fraud cases, i.e., 10% fraudulent observations



Re-sampling

- Borderline-SMOTE (Han et al., 2005)
- ADASYN (He et al., 2008)
- Safe-level SMOTE (Bunkhumpornpat et al., 2009)
- Density-based SMOTE (Bunkhumpornpat et al., 2011)
- MWMOTE (Barua et al., 2014)

Re-sampling

- When under- or oversampling, **class priors** are altered
  - As a result, classifier predictions, i.e., probability estimates, will be biased
- Probability estimates can be adjusted as follows (Saerens et al., 2002)

$$p(C_i|x) = \frac{\frac{p(C_i)}{p_t(C_i)} p_t(C_i|x)}{\sum_{j=1}^{2} \frac{p(C_j)}{p_t(C_j)} p_t(C_j|x)}$$

  - C_i is class i
  - $p_t(C_i|x)$ is probability estimated on over- or undersampled training data set
  - $p_t(C_i)$ is prior probability of class C_i on over- or undersampled training data set
  - $p(C_j)$ represents original priors (e.g., 99%, 1%)

Re-sampling

- $p(C_i|x) = \frac{\frac{p(C_i)}{p_t(C_i)} p_t(C_i|x)}{\sum_{j=1}^{2} \frac{p(C_j)}{p_t(C_j)} p_t(C_j|x)}$

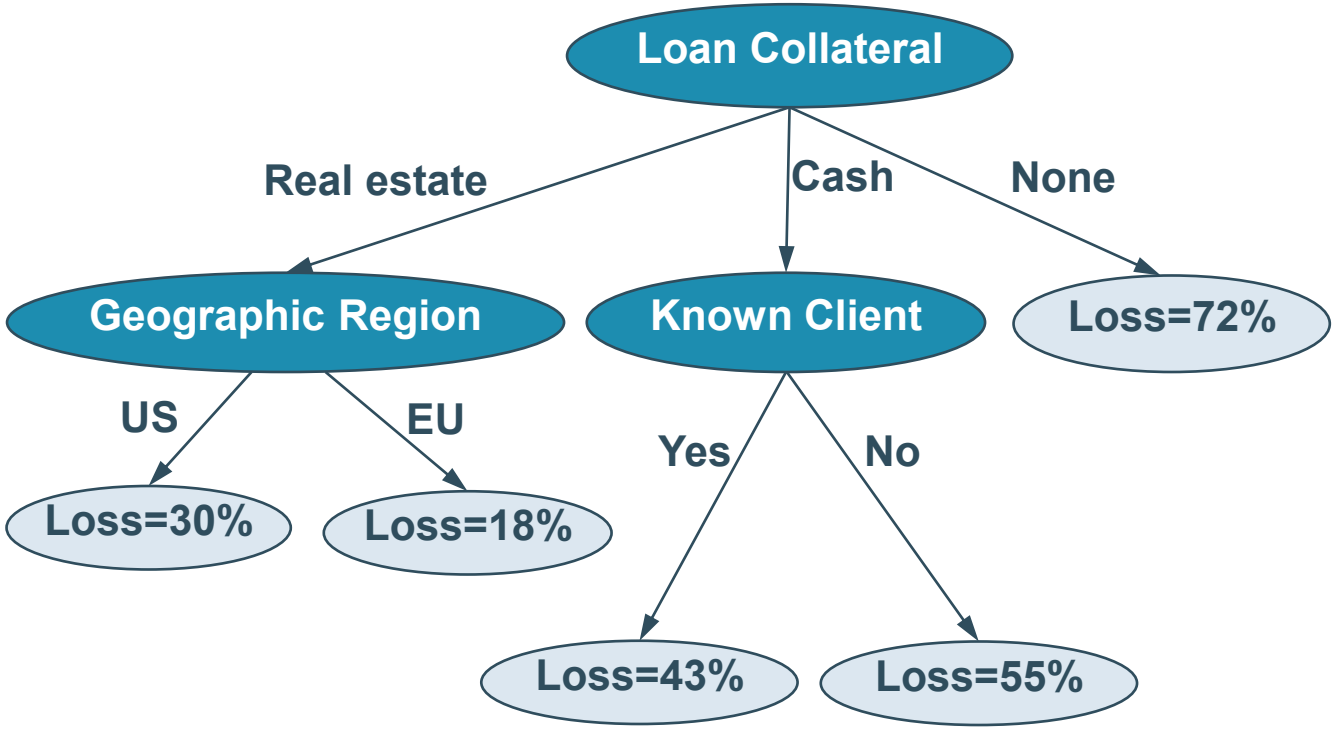
- Example with:
  - $p(C_1) = 0.01$
  - $p(C_2) = 0.99$
  - $p_t(C_1) = 0.20$
  - $p_t(C_2) = 0.80$

Obs.	Posteriors using resampled data		Posteriors re-calibrated to original data	
	P(Fraud)	P(No Fraud)	P(Fraud)	P(No Fraud)
1	0,1	0,9	0,004	0,996
2	0,3	0,7	0,017	0,983
3	0,5	0,5	0,039	0,961
4	0,6	0,4	0,057	0,943
5	0,85	0,15	0,186	0,814
6	0,9	0,1	0,267	0,733

3. Regression trees

Regression trees

- If the target variable is continuous:
  - Loss Given Default (LGD)
  - Customer Lifetime Value (CLV)
  - …
- Splitting decision
  - Impurity?



FYI: Cash collateral is cash, negotiable instruments, documents of title, securities, deposit accounts, and other cash equivalents
https://www.wise-geek.com/what-are-the-different-types-of-cash-collateral.htm

Splitting decision

- Mean Squared Error

$$I(N) = \frac{1}{N}\sum_{i=1}^{N}(Y_i - \bar{Y})^2, \text{ with N} = \text{number of observations in node}$$

- ANOVA/F-test

$$F = \frac{SS_{between}/(B-1)}{SS_{within}/(N-B)} \sim F_{B-1,N-B}$$

Compute the p-value
Low p-value indicates good split!

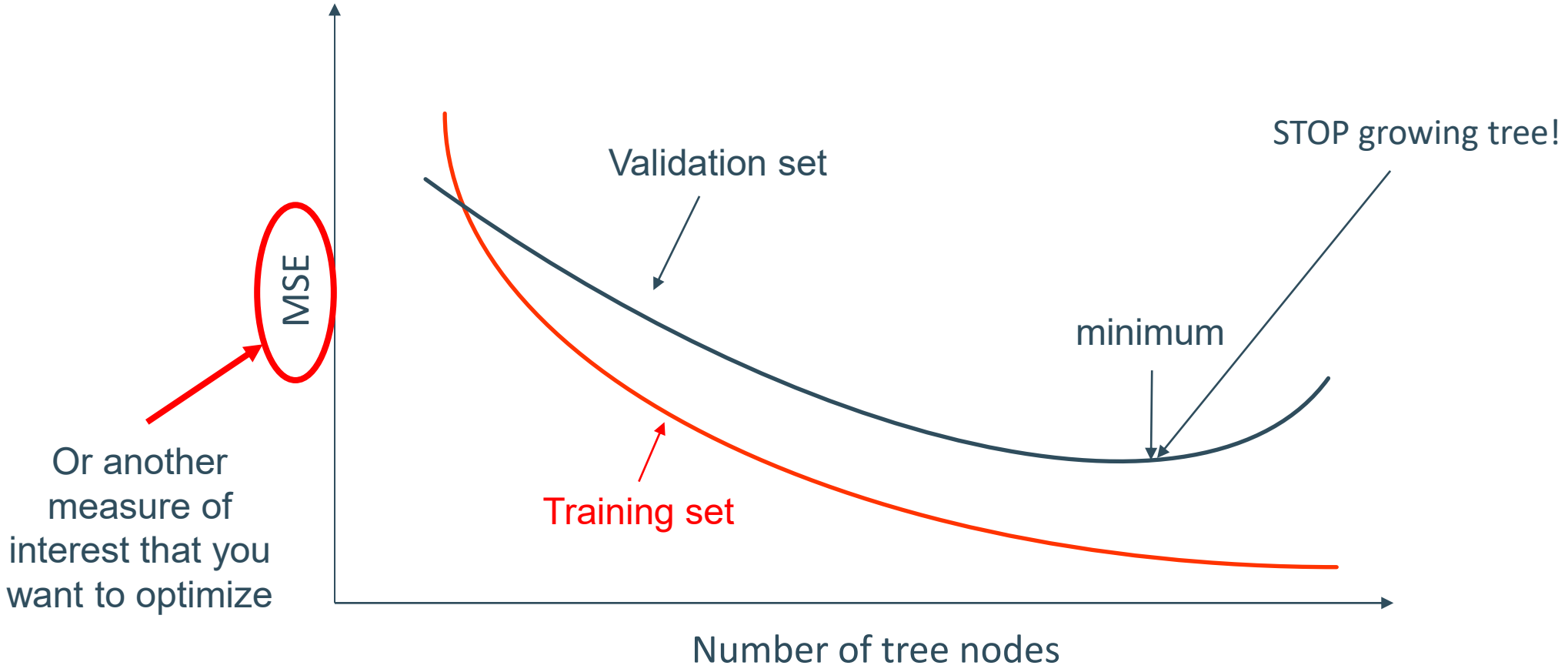
$$SS_{between} = \sum_{b=1}^{B} N_b(\bar{Y}_b - \bar{Y})^2,$$

with B = number of branches, N_b number of observations in branch b, $\bar{Y}_b$ the mean in branch b

$$SS_{within} = \sum_{b=1}^{B}\sum_{i=1}^{N_b}(Y_{bi} - \bar{Y}_b)^2, \text{ with } \bar{Y}_b \text{ the mean in branch b}$$

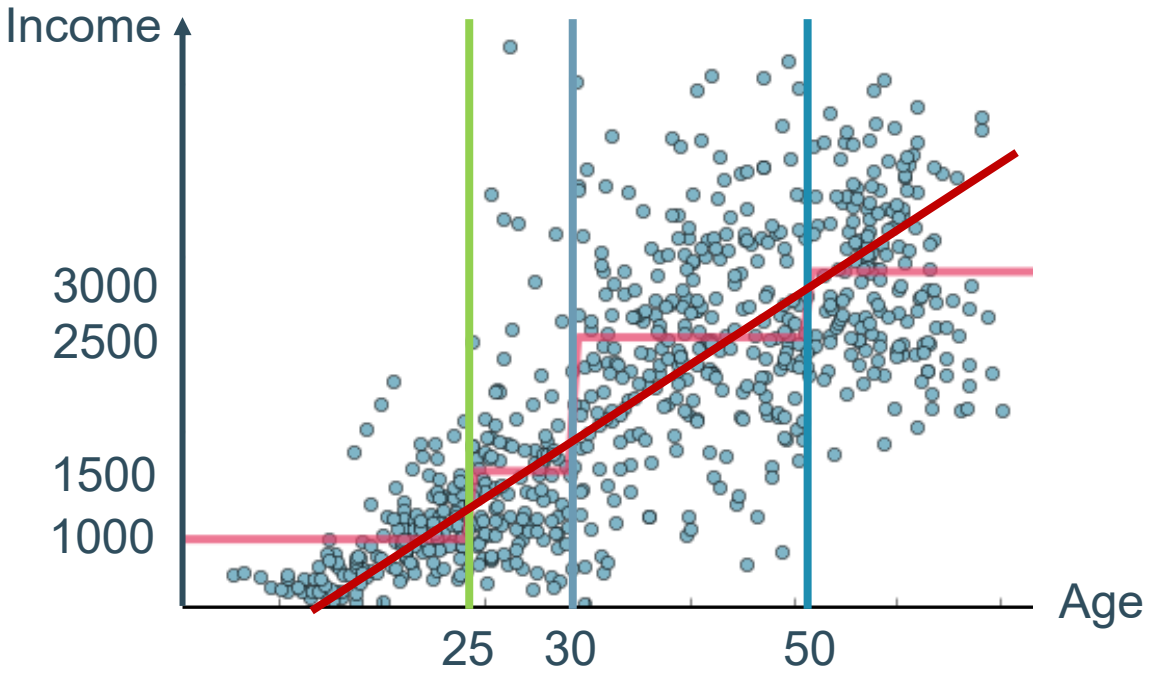
Stopping decision

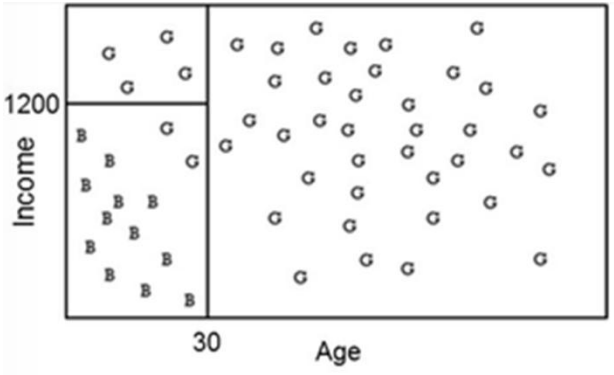
Remember: hyperparameter tuning!

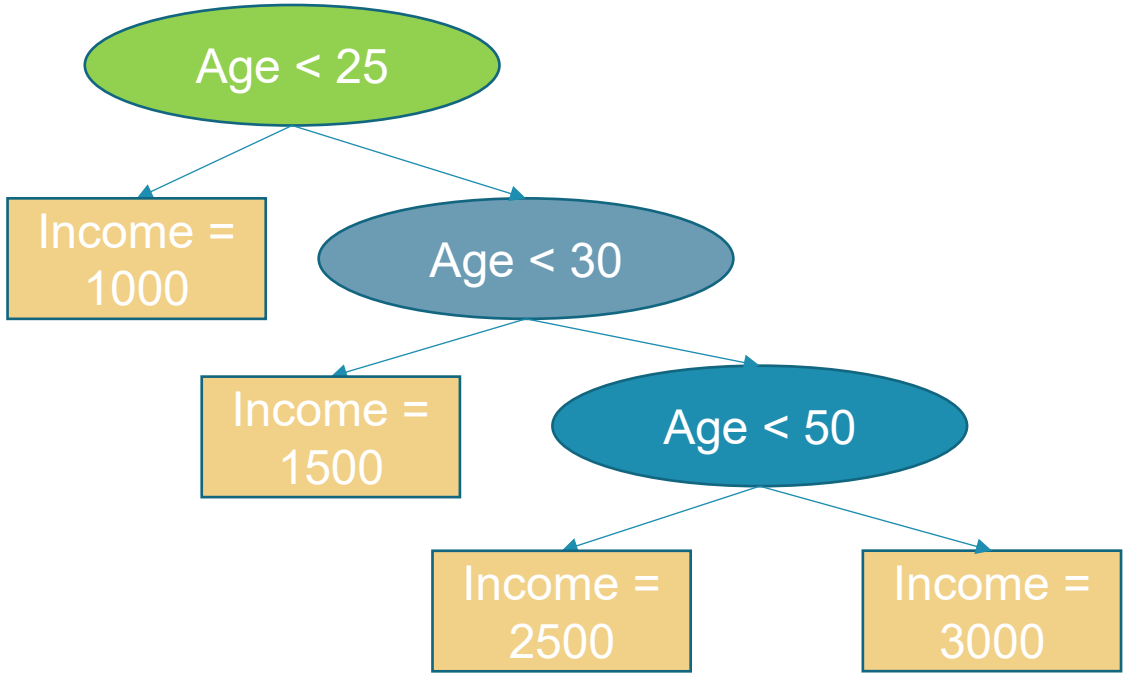


Regression trees

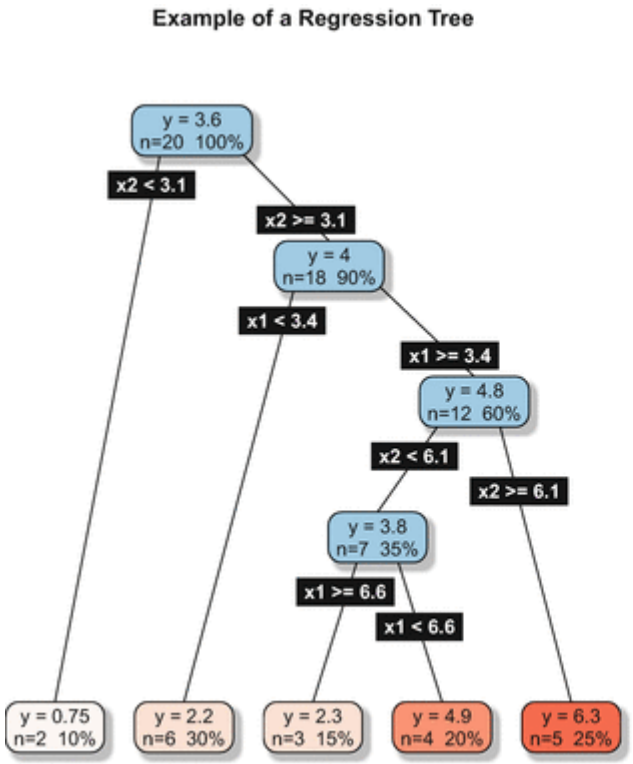
- Regression ‘curve’?
- Example:
  - Tree fitted with CART
  - Linear regression model: **y = a + bx**

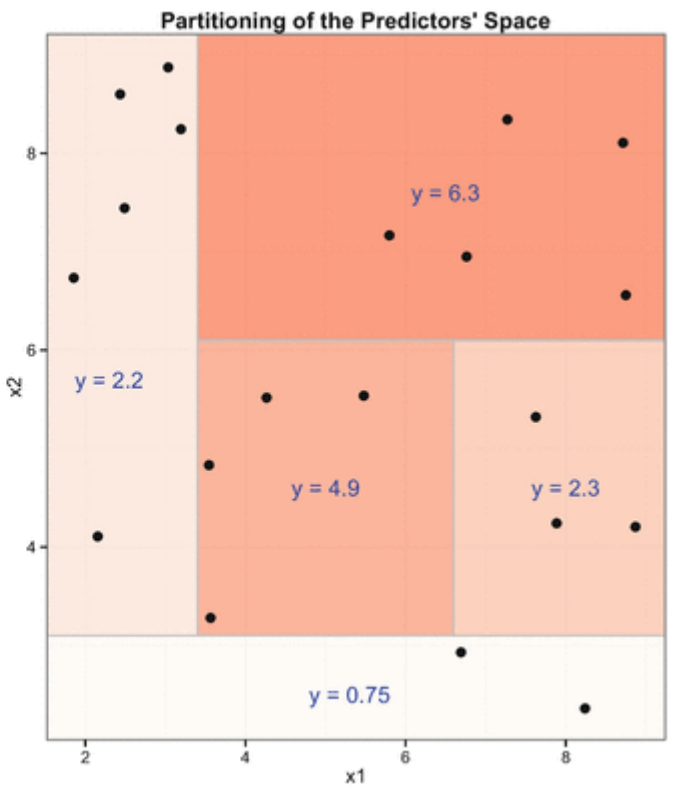






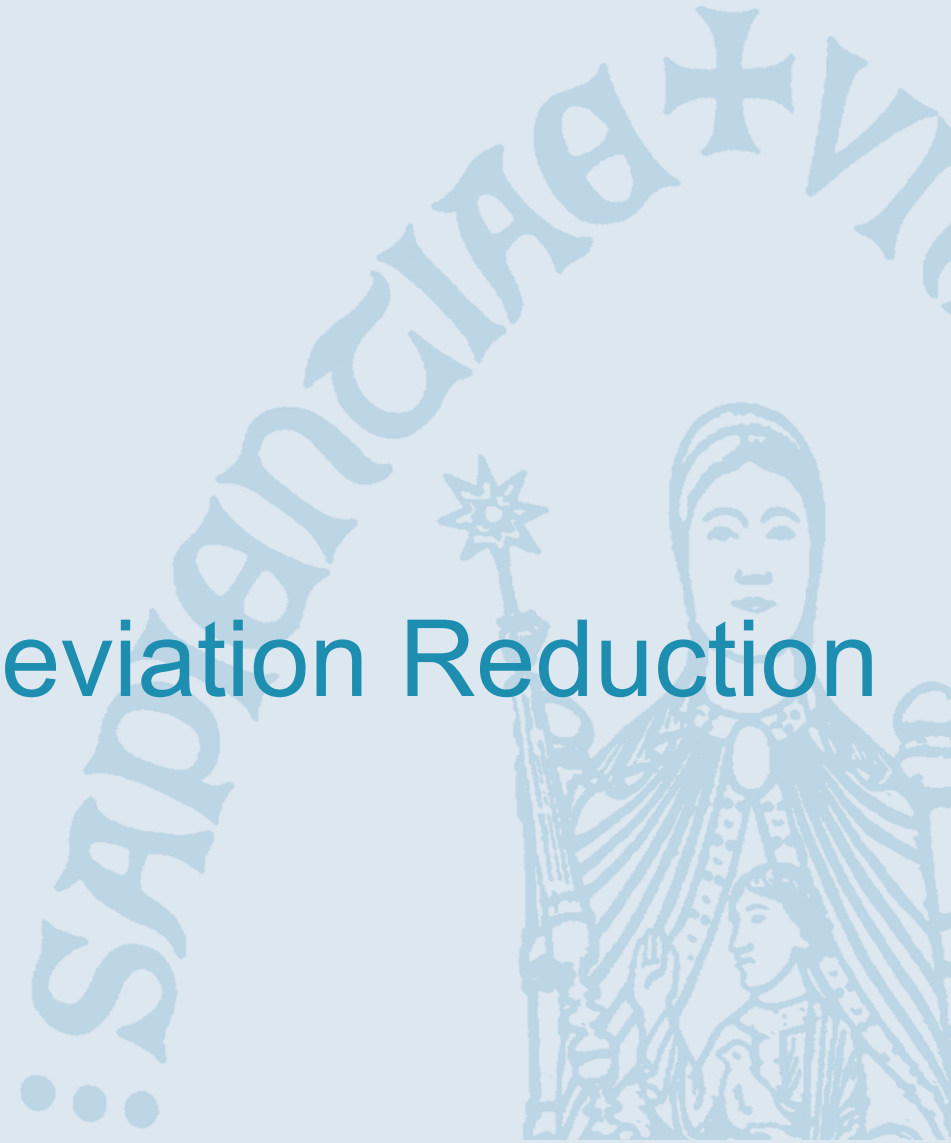
Regression trees





https://link.springer.com/referenceworkentry/10.1007%2F978-1-4899-7687-1_717

Self-study: ID3 with Standard Deviation Reduction



ID3 with SDR

- The **ID3 algorithm** for building decision trees by J.R. Quinlan employs a top-down, **greedy** search through the space of possible branches

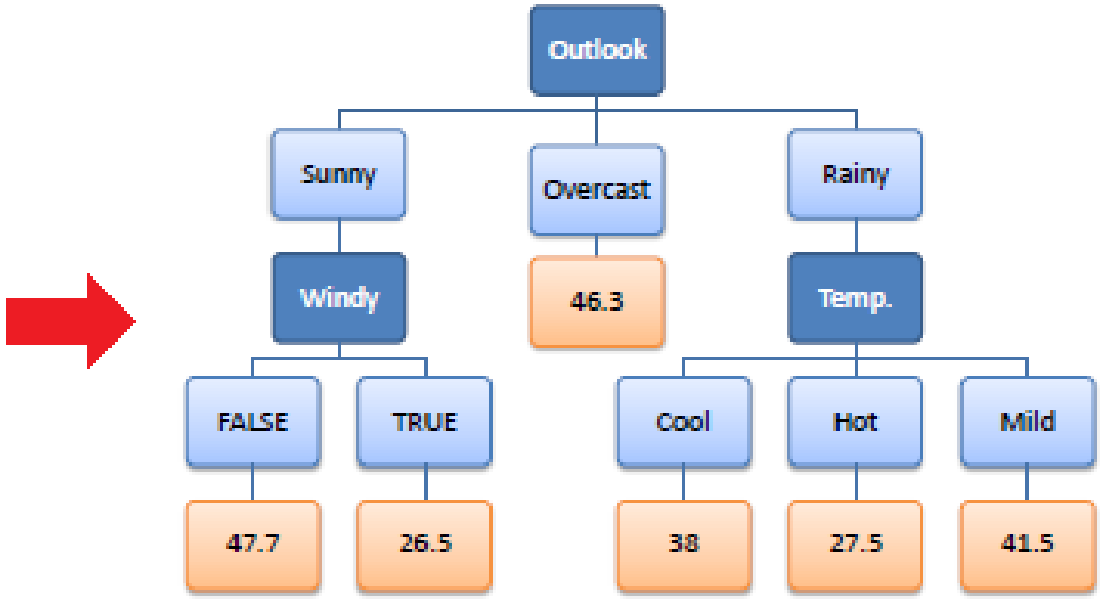
- The ID3 algorithm can be used to construct a decision tree for **regression** by replacing ***Information Gain*** with ***Standard Deviation Reduction***

- Standard deviation reduction:
  - The standard deviation reduction is based on the (weighted) decrease in standard deviation after a dataset is split on an attribute
  - Constructing a decision tree is all about **finding the attribute split** returning the highest standard deviation reduction (i.e., the most homogeneous branches)

ID3 with SDR

- Example:

Predictors				Target
Outlook	Temp.	Humidity	Windy	Hours Played
Rainy	Hot	High	False	25
Rainy	Hot	High	True	30
Overcast	Hot	High	False	48
Sunny	Mild	High	False	46
Sunny	Cool	Normal	False	52
Sunny	Cool	Normal	True	23
Overcast	Cool	Normal	True	43
Rainy	Mild	High	False	35
Rainy	Cool	Normal	False	38
Sunny	Mild	Normal	False	48
Rainy	Mild	Normal	True	48
Overcast	Mild	High	True	52
Overcast	Hot	Normal	False	44
Sunny	Mild	High	True	30



ID3 with SDR

Hours Played
25
30
46
45
52
23
43
35
38
46
48
52
44
30



$$\textbf{\textit{Count}} = n = 14$$

$$\textbf{\textit{Average}} = \bar{x} = \frac{\sum x}{n} = 39.8$$

$$\textbf{\textit{Standard Deviation}} = S = \sqrt{\frac{\sum (x - \bar{x})^2}{n}} = 9.32$$

$$\textbf{\textit{Coeffeicient of Variation}} = CV = \frac{S}{\bar{x}} * 100\% = 23\%$$

- Standard Deviation (**S**) is for tree building (branching)
- Coefficient of Variation (**CV**) is used to decide **when to stop branching**
  - We can use Count (**n**) as well, e.g., to stop branching when n < threshold
- Average ($\bar{x}$) is the predicted value in the leaf nodes

https://www.saedsayad.com/decision_tree_reg.htm
https://datacollection.jrc.ec.europa.eu/wordef/coefficient-of-variation

ID3 with SDR

- ***Step 1***: The standard deviation of the target is calculated

Standard deviation (Hours Played) = 9.32

Hours Played
25
30
46
45
52
23
43
35
38
46
48
52
44
30



$$\boldsymbol{Count} = n = 14$$

$$\boldsymbol{Average} = \bar{x} = \frac{\sum x}{n} = 39.8$$

$$\boldsymbol{Standard\ Deviation} = S = \sqrt{\frac{\sum (x - \bar{x})^2}{n}} = 9.32$$

$$\boldsymbol{Coeffeicient\ of\ Variation} = CV = \frac{S}{\bar{x}} * 100\% = 23\%$$

ID3 with SDR

- ***Step 2***: The dataset is split on the different attributes
  - The standard deviation for each branch is calculated
  - The ***resulting*** standard deviation is subtracted from the standard deviation before the split
  - The result is the ***standard deviation reduction (SDR)***

$$S(T,X) = \sum_{c \in X} P(c)S(c)$$

		Hours Played (StDev)	Count
Outlook	Overcast	3.49	4
	Rainy	7.78	5
	Sunny	10.87	5
			14

S(Hours, Outlook) = **P**(Sunny)***S**(Sunny) + **P**(Overcast)***S**(Overcast) + **P**(Rainy)***S**(Rainy)

= (4/14)*3.49 + (5/14)*7.78 + (5/14)*10.87

= 7.66

$$SDR(T,X) = S(T) - S(T,X)$$

SDR(Hours , Outlook) = **S**(Hours) – **S**(Hours, Outlook)

= 9.32 – 7.66 = 1.66

		Hours Played (StDev)
Outlook	Overcast	3.49
	Rainy	7.78
	Sunny	10.87
SDR=1.66		

		Hours Played (StDev)
Temp.	Cool	10.51
	Hot	8.95
	Mild	7.65
SDR= 0.48		

		Hours Played (StDev)
Humidity	High	9.36
	Normal	8.37
SDR=0.28		

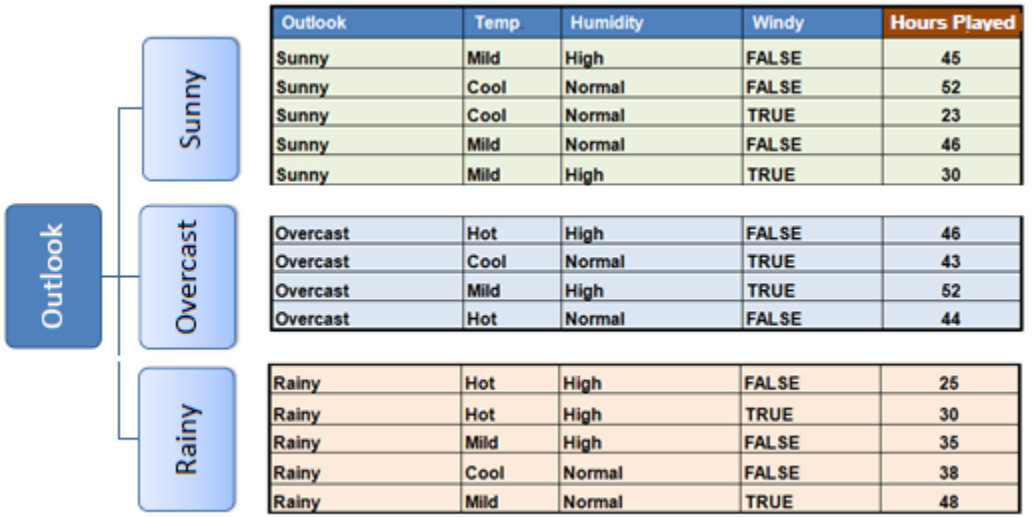
		Hours Played (StDev)
Windy	False	7.87
	True	10.59
SDR=0.29		

ID3 with SDR

- ***Step 3***: The attribute with the largest standard deviation reduction is chosen for the decision node

★		Hours Played (StDev)
Outlook	Overcast	3.49
	Rainy	7.78
	Sunny	10.87
SDR=1.66		

ID3 with SDR



Outlook	Temp	Humidity	Windy	Hours Played
Sunny	Mild	High	FALSE	45
Sunny	Cool	Normal	FALSE	52
Sunny	Cool	Normal	TRUE	23
Sunny	Mild	Normal	FALSE	46
Sunny	Mild	High	TRUE	30
Overcast	Hot	High	FALSE	46
Overcast	Cool	Normal	TRUE	43
Overcast	Mild	High	TRUE	52
Overcast	Hot	Normal	FALSE	44
Rainy	Hot	High	FALSE	25
Rainy	Hot	High	TRUE	30
Rainy	Mild	High	FALSE	35
Rainy	Cool	Normal	FALSE	38
Rainy	Mild	Normal	TRUE	48

- ***Step 4***: The dataset is divided based on the values of the selected attribute. This process is run recursively on the non-leaf branches, until all data is processed
  - Iteratively add branches until reaching some stopping (or termination) criteria.
  - E.g.: when coefficient of variation (**CV**) for a branch becomes smaller than a certain threshold (e.g., 10%) **and/or** when too few instances (**n**) remain in the branch (e.g., 3).

Q&A